

國立陽明交通大學
智慧系統與應用研究所
碩士論文

Institute of Intelligent Systems
National Yang Ming Chiao Tung University
Master Thesis

FieldAnchor：農田無人機—衛星資料集與無 GPS 導航局
部視覺重定位之硬式均值漂移解碼
FieldAnchor: A Farmland UAV–Satellite Dataset and Hard
Mean-Shift Decoding for GPS-less Navigation

研究生：李宇弘（Yu-Hong Lee）
指導教授：謝君偉（Jun-Wei Hsieh）

中華民國一一五年八月

August 2026

FieldAnchor：農田無人機—衛星資料集與無 GPS 導航局

部視覺重定位之硬式均值漂移解碼

FieldAnchor: A Farmland UAV–Satellite Dataset and Hard

Mean-Shift Decoding for GPS-less Navigation

研 究 生：李宇弘

Student：Yu-Hong Lee

指導教授：謝君偉 博士

Advisor：Dr. Jun-Wei Hsieh

國立陽明交通大學

智慧系統與應用研究所

碩士論文

A Thesis

Submitted to Institute of Intelligent Systems

College of Artificial Intelligence

National Yang Ming Chiao Tung University

in partial Fulfillment of the Requirements

for the Degree of Master

in

Artificial Intelligence

August 2026

Taiwan, Republic of China

中華民國一一五年八月

致謝

兩年的研究所生涯即將告一段落，回顧這段求學歷程，首先要誠摯感謝我的指導教授 謝君偉教授。在研究進行的過程中，教授持續透過定期會議掌握研究進度，並針對研究問題、實驗設計與論文撰寫給予許多寶貴的建議。當研究遭遇瓶頸或實驗結果不如預期時，教授也會引導我重新檢視問題，從研究動機、方法設計到實驗設定逐步分析，協助我釐清問題的核心。透過這段時間的訓練，我逐漸學會在面對複雜問題時，先將問題拆解，再依序驗證可能的原因與解決方案，而不是急於尋求答案。這樣的思考方式不僅運用於研究與程式開發，也使我在處理各種挑戰時更加有條理。感謝教授一路以來的指導與督促，使我能順利完成研究，也培養了獨立思考、解決問題及面對壓力的能力。

此外，也要感謝研究室的所有夥伴，讓忙碌的研究生活增添了許多溫暖與值得珍藏的回憶。感謝東琳同學在我剛進入研究室、尚未熟悉環境時，熱心分享相關經驗並提供協助，使我能更快適應研究所的生活。感謝晉翔與博云在論文進入最後階段時，陪伴我度過許多長時間留在實驗室的日子。無論是一起用餐、討論研究，或是在疲憊時到戶外走走、聊聊近況，這些看似平凡的日常，都成為支撐我繼續努力的重要力量。同時，也感謝研究室其他同學平時的關心與陪伴，大家一起聊天、運動、聚餐及玩桌遊的時光，讓我能在此緊湊的研究步調中獲得短暫的放鬆。因為有各位的陪伴，這段研究所生活不只有實驗與論文，也留下了許多真誠而美好的回憶。

最後，我要將最深的感謝獻給我的家人。一路以來，家人始終尊重我的選擇，並給予我充分的信任與支持。在研究進度落後、實驗反覆失敗或對未來感到迷惘時，家人的關心總能讓我重新整理情緒，以更加穩定的心情繼續面對眼前的挑戰。他們或許無法完全理解研究中的技術內容，卻總是願意耐心聆聽我的煩惱，提醒我要照顧身體，也讓我知道無論最後的結果如何，家永遠是能夠安心依靠的地方。正因為有家人在背後默默支持，我才能專心完成學業，並堅持走到最後。謹以此篇論文獻給所有曾經幫助、鼓勵與陪伴我的人，感謝你們共同成就了這段珍貴的研究所旅程。

FieldAnchor：農田無人機—衛星資料集與無 GPS 導航局 部視覺重定位之硬式均值漂移解碼

學生：李宇弘

指導教授：謝君偉 教授

國立陽明交通大學智慧系統與應用研究所

摘要

全球定位系統（Global Positioning System, GPS）是無人機取得絕對位置的重要來源；當 GPS 受干擾或遭拒止時，無人機需藉由視覺資訊將當前影像對應至地理參考衛星圖。農田具有重複田區、平行田埂與相似植被紋理，使鄰近候選容易產生混淆。本文提出 FieldAnchor 無人機—衛星資料集，並提出 FieldAnchor Local Relocalization (FieldAnchor-LR) 局部視覺重定位框架，在已知預定航線與受控粗略位置條件下進行定位。預定航線由起點、依序定義的參考點與終點構成；每次定位先以粗略位置建立完整 6×6 （36 個）局部衛星候選幾何，再依因果前向方向進行硬式候選限制，只保留前方 3×6 （18 個）候選，後方 18 個候選在在線投影與相似度計算前即排除。保留候選經凍結的 MobileCLIP2-S2 與雙分支投影器取得跨視角特徵，接著以均值漂移（Mean-Shift, MS）整合空間響應並估計連續視覺位置與不確定度。門控循環單元（Gated Recurrent Unit, GRU）融合最近三幀影像特徵、衛星脈絡及前一時刻狀態，估計速度、加速度與量測資訊；二階慣性多項式形成下一步運動預測，最後由不確定度感知卡爾曼濾波（Kalman Filter, KF）融合運動預測與視覺量測，得到連續定位結果。本文標題中的「Hard」僅指 Mean-Shift 解碼前由完整 6×6 幾何硬式保留前向 3×6 候選的限制，並非另一種獨立的 Mean-Shift 演算法。

關鍵字：無人機定位、衛星影像、跨視角地理定位、均值漂移、時序定位、門控循環單元、卡爾曼濾波。



FieldAnchor: A Farmland UAV–Satellite Dataset and Hard Mean-Shift Decoding for GPS-less Navigation

Student: Yu-Hong Lee

Advisor: Dr. Jun-Wei Hsieh

Institute of Intelligent Systems
College of Artificial Intelligence
National Yang Ming Chiao Tung University

Abstract.

Global Positioning System (GPS) measurements are an important source of absolute position for unmanned aerial vehicles (UAVs). When GPS is disrupted or denied, visual observations can be matched to a georeferenced satellite map to recover position. Repetitive field boundaries and similar vegetation make this difficult in farmland because nearby satellite candidates can appear alike. This thesis introduces FieldAnchor, a UAV–satellite dataset, and proposes the FieldAnchor Local Relocalization (FieldAnchor-LR) framework under a known planned route and a controlled coarse-position condition. The planned route is represented by a start point, ordered predefined reference points, and a destination. For each query, a complete 6×6 local satellite geometry containing 36 candidates is first established around the coarse position. A hard forward candidate restriction is then applied before online visual scoring; only the route-aligned forward 3×6 subset containing 18 candidates is retained, while the rear 18 candidates are excluded before satellite projection and similarity computation. A frozen MobileCLIP2-S2 encoder with view-specific projectors forms cross-view features for the retained candidates, and Mean-Shift (MS) aggregates the spatial response to estimate a continuous visual position and uncertainty. A three-frame Gated Recurrent Unit (GRU) combines recent image features, satellite context, and the previous state to estimate motion and measurement information. A second-order inertial polynomial predicts the next displacement, and an uncertainty-aware Kalman Filter (KF) fuses the motion prediction with the visual measurement to obtain the final continuous position. The term “Hard” refers only to the explicit forward candidate restriction applied before Mean-Shift decoding; it does not denote a separate variant of the Mean-Shift algorithm.

Keywords: UAV localization, satellite imagery, cross-view geo-localization, Mean-Shift, temporal localization, Gated Recurrent Unit, Kalman filtering.



目錄

摘要	i
Abstract.....	iii
目錄	v
List of Figures.....	vii
List of Tables.....	viii
Chapter 1 Introduction	1
1.1 Research Background and Motivation	1
1.2 Problem Definition	1
1.3 Research Contributions	2
Chapter 2 Related Work.....	3
2.1 Cross-View Geo-Localization	3
2.2 UAV-Satellite Localization	3
2.3 Spatial Response Decoding	3
2.4 Temporal State Estimation and Filtering	4
Chapter 3 FieldAnchor Dataset and Benchmark Protocol	4
3.1 Data Collection and Route Split	4
3.2 Fixed Satellite Anchor Lattice	6
3.3 Controlled Local Candidate Construction	7
3.4 Scope and Data Release.....	9
3.5 Acquisition Filtering and Image Preprocessing	10
3.6 Map Registration and Metric Coordinate System	10
3.7 Benchmark Package, Privacy Boundary, and Versioning	11
Chapter 4 Proposed Method.....	12
4.1 Overview	12
4.2 UAV-Satellite Visual Representation	13
4.3 Hard Forward 3×6 Candidate Restriction.....	14
4.4 Mean-Shift Visual Localization.....	15
4.5 Continuous Route Coordinates and Waypoint Representation	18
4.6 Three-Frame Gated Recurrent Unit Temporal Modeling.....	19

4.7	Second-Order Inertial Motion Prediction	21
4.8	Visual Measurement Correction and Uncertainty Estimation	22
4.9	Uncertainty-Aware Kalman Filtering	22
4.10	Training Objective	24
4.11	End-to-End Inference	27
4.12	Computational Complexity and Reproducibility	29
Chapter 5	Experiments and Results	29
5.1	Experimental Settings and Metrics	29
5.2	MS and Weighted-Centroid Accuracy–Runtime Trade-off	30
5.3	FieldAnchor-LR Component Ablation	30
5.4	Hard Forward Candidate Restriction	31
5.5	Temporal Window Length	32
5.6	Second-Order Inertial Motion Model	32
5.7	Kalman Filtering and Measurement Variance	33
5.8	Route-Wise Final Localization Results	34
5.9	Comparison with Existing UAV–Satellite Localization Methods	35
5.10	Backbone Accuracy–Runtime Trade-off on NVIDIA Jetson Orin NX	37
5.11	Qualitative Trajectory Analysis	38
Chapter 6	Conclusion	41
References		42

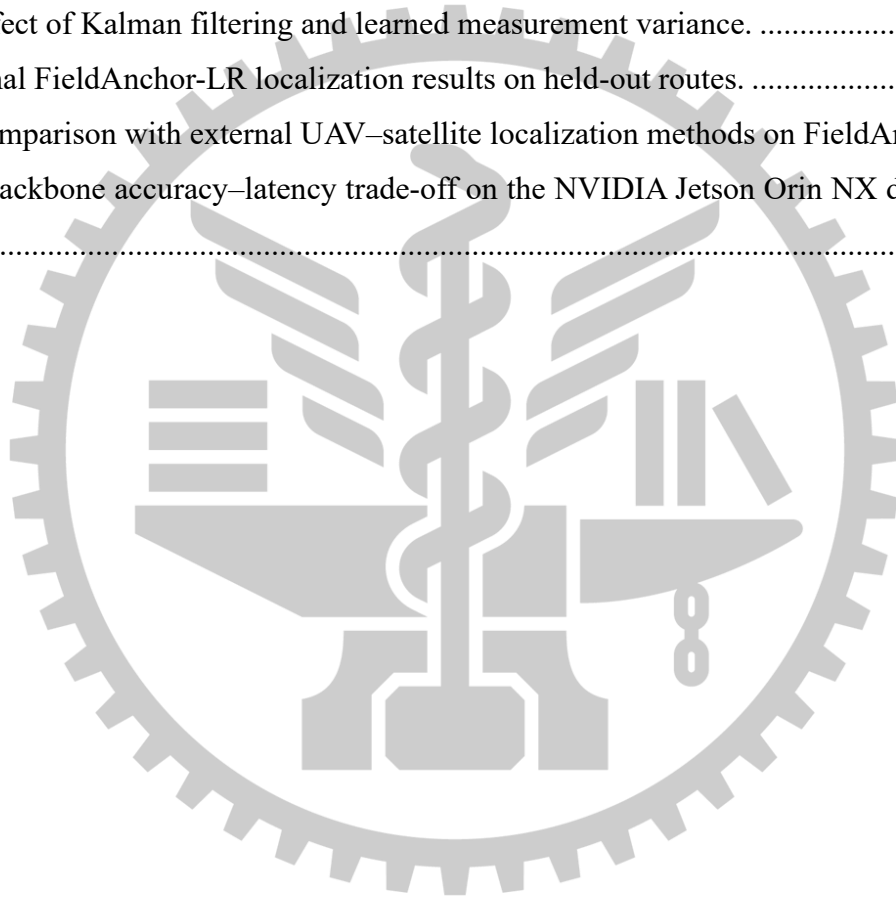
List of Figures

Figure 1. Route-held-out FieldAnchor split over the agricultural orthomosaic.	5
Figure 2. Permanent satellite lattice and a controlled local candidate window.	7
Figure 3. Reference-anchor slot distributions in the local candidate grid.	8
Figure 4. Overview of the FieldAnchor-LR local relocalization framework.	13
Figure 5. Hard forward 3×6 candidate restriction.	15
Figure 6. Mean-Shift (MS) continuous visual localization flow.	16
Figure 7. Three-frame GRU and second-order inertial prediction.	21
Figure 8. Uncertainty-aware Kalman fusion.	24



List of Tables

Table 1. FieldAnchor route statistics after flight-height filtering.....	6
Table 2. Accuracy and coordinate-aggregation runtime comparison between weighted centroid and MS.....	33
Table 3. Progressive ablation of the FieldAnchor-LR localization framework.....	34
Table 4. Comparison of full 6×6 and forward 3×6 candidate search	35
Table 5. Effect of the temporal input window.	35
Table 6. Ablation of the inertial motion prediction.	36
Table 7. Effect of Kalman filtering and learned measurement variance.	37
Table 8. Final FieldAnchor-LR localization results on held-out routes.	37
Table 9. Comparison with external UAV–satellite localization methods on FieldAnchor.	38
Table 10. Backbone accuracy–latency trade-off on the NVIDIA Jetson Orin NX development platform.	40



Chapter 1 Introduction

1.1 Research Background and Motivation

Global Navigation Satellite System (GNSS) measurements are a common source of absolute position for an unmanned aerial vehicle (UAV), but their availability and accuracy can deteriorate under signal blockage, interference, or denied environments. A complementary visual localization module can recover position information by matching the current UAV observation to a georeferenced satellite or orthophoto map. The problem is particularly difficult in agricultural areas, where repeated field boundaries, irrigation channels, access roads, and similar crop textures can generate several visually plausible satellite responses within a compact neighborhood.

Cross-view geo-localization has progressed from global image retrieval toward UAV self-positioning and metric localization. Retrieval-oriented methods such as CVM-Net [1], SAFA [2], TransGeo [3], and Sample4Geo [4] learn cross-view representations, while UAV-oriented benchmarks and systems including University-1652 [5], SUES-200 [6], DenseUAV [7], Game4Loc [8], Bearing-UAV [9], and InfoGeo [10] address aerial-to-satellite matching under increasingly realistic conditions. Recent work further studies noisy correspondence, unified drone-satellite-ground matching, multi-view foundation models, geometry-guided geo-localization, and direct UAV ego-localization [16]–[20]. These advances motivate a local navigation setting in which visual evidence, route structure, temporal motion, and an explicit state estimator are combined rather than treated as isolated independent-frame decisions.

1.2 Problem Definition

This thesis studies route-conditioned local visual relocalization rather than unconstrained global map search. The mission is assumed to provide a known planned route defined by the start point, ordered predefined route reference points, and destination. Each localization query is associated with one reference point on this planned route and a controlled coarse local position estimate that is used only to establish a compact satellite candidate neighborhood. The predefined reference point is retained separately as a supervision and evaluation target; it is not concatenated with the UAV image embedding, satellite visual embedding, or recurrent feature vector, and it is not reported as the localization output. The thesis therefore evaluates visual refine

ment around a known planned route while keeping the route reference annotations separate from the learned visual and recurrent representations.

Under this controlled local-search condition, the objective is to refine an imperfect coarse location using UAV–satellite visual evidence while preserving temporal consistency. The reported experiments therefore evaluate local visual relocalization along a prescribed route, including metric accuracy, temporal stability, robustness to ambiguous satellite responses, and the computational trade-off of forward candidate restriction.

1.3 Research Contributions

The main contributions of this thesis are summarized as follows: (1) FieldAnchor, a real-flight farmland UAV–satellite dataset with a registered orthomosaic and route-held-out evaluation split; (2) a controlled local-search benchmark based on predefined planned-route reference points, together with a hard forward candidate restriction that reduces a complete 6×6 local geometry from 36 candidates to the route-aligned forward 3×6 subset of 18 candidates before on-line projection and similarity scoring; (3) a Mean-Shift (MS) decoder that converts the retained candidate responses into a continuous visual position and uncertainty; and (4) a three-frame Gated Recurrent Unit (GRU), second-order inertial predictor, learned measurement variance, and external Kalman Filter (KF) that improve temporal state estimation while keeping the reference points used for supervision and evaluation separate from the model input representation.

Chapter 2 Related Work

2.1 Cross-View Geo-Localization

Cross-view geo-localization determines geographic correspondence between observations acquired from substantially different viewpoints. CVM-Net [1] and SAFA [2] established learned cross-view feature matching and spatial aggregation, while TransGeo [3] introduced transformer-based representations for global geo-localization. Sample4Geo [4] demonstrated the value of geographically and visually hard negatives. More recent studies broaden this formulation: PAUL [16] targets noisy cross-view correspondence, UniGeoRS [17] unifies satellite, drone, and ground-view localization in one benchmark, GeoBridge [18] incorporates semantic anchors across multiple views and modalities, and Geo2 [20] introduces geometry-guided cross-view localization. These methods strengthen cross-view representation learning, but they primarily emphasize retrieval or single-observation geo-localization rather than the causal temporal state required by the route-conditioned estimator studied in this thesis.

2.2 UAV–Satellite Localization

UAV-specific datasets and methods have progressively reduced the gap between image retrieval and practical self-positioning. University-1652 [5] provides a multi-view drone benchmark, SUES-200 [6] studies cross-view changes across scene types and flight heights, DenseUAV [7] targets low-altitude UAV self-positioning with dense sampling and distance-aware evaluation, and Game4Loc [8] extends UAV geo-localization to continuous areas with metric localization error. In 2026, Bearing-UAV [9] directly predicts absolute position and heading from neighboring aerial–satellite features, InfoGeo [10] improves cross-view generalization through object-centric information modeling, and PiLoT [19] performs UAV ego- and target geo-localization by registering live video against a georeferenced 3D map. These studies motivate the transition from image-level retrieval toward navigation-oriented localization, while FieldAnchor-LR focuses specifically on route-conditioned local UAV–satellite relocalization with causal temporal state propagation.

2.3 Spatial Response Decoding

Mean shift is a classical non-parametric mode-seeking procedure that iteratively moves a point toward a locally dense region under a kernel-weighted distribution [11]. Hsieh et al. [15]

further demonstrate a mean-shift-based optimization mechanism in differentiable architecture search, where iterative shifting within a selected bandwidth is used to smooth the search landscape and improve stability. Although that work addresses neural architecture search rather than UAV localization, it provides a recent example of using mean shift as a differentiable iterative refinement mechanism. In FieldAnchor-LR, Mean-Shift (MS) is instead applied to georeferenced satellite candidate centers, using the visual posterior as response mass to obtain a continuous visual position. The final implementation does not snap the result to a discrete satellite anchor; shifted modes contribute through density-based weights, and the same response distribution is used to derive visual uncertainty before temporal fusion.

2.4 Temporal State Estimation and Filtering

Temporal localization requires a state representation that can combine current visual evidence with recent motion. The Gated Recurrent Unit (GRU), introduced as a recurrent gating mechanism for sequence modeling [13], provides a compact way to preserve relevant history without using a long explicit image stack. In parallel, Kalman filtering provides a principled recursive framework for combining a motion prior and a noisy measurement [14]. The proposed system uses a three-frame GRU to estimate velocity, acceleration, heading residual, visual correction, and measurement variance, and then applies an external KF whose measurement covariance and update magnitude are adjusted according to the visual uncertainty. This division keeps learned temporal representation and explicit state estimation separate and inspectable.

Chapter 3 FieldAnchor Dataset and Benchmark Protocol

3.1 Data Collection and Route Split

FieldAnchor contains real UAV imagery acquired over agricultural land together with one registered satellite orthomosaic. The retained frames are organized into three flight routes, with partial spatial overlap among the routes. Route A is used for training, while Routes B and C are used for testing. This route-based protocol evaluates the localization performance of the model trained on Route A when applied to Routes B and C within the same mapped agricultural environment.

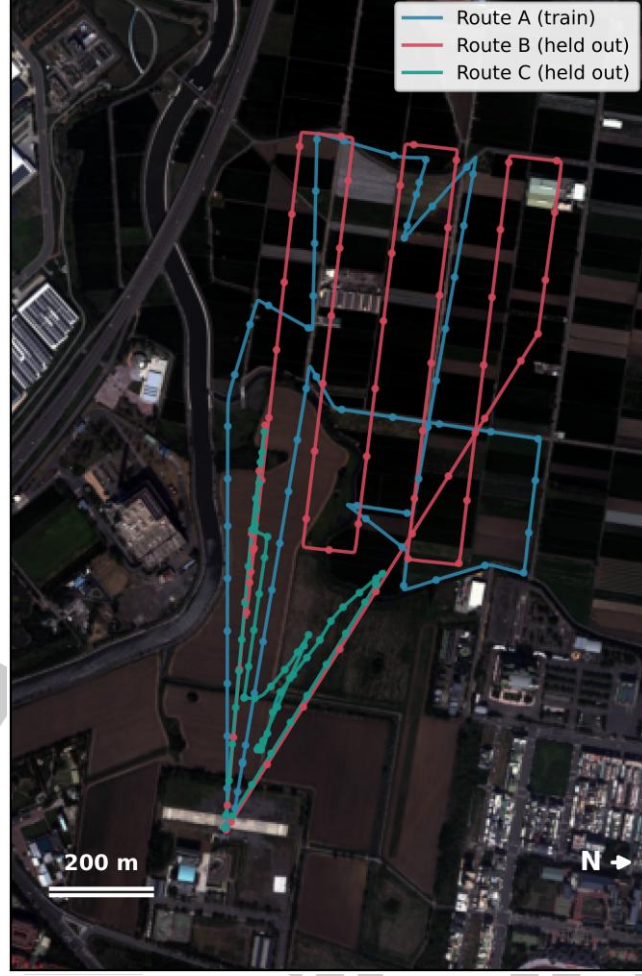


Figure 1. Route-held-out FieldAnchor split over the agricultural orthomosaic. The route-held-out split uses Route A for training, whereas Routes B and C are reserved for held-out evaluation. The separation is performed at the route level rather than by randomly mixing neighboring frames, so the held-out results measure localization on flight segments that are not part of the training trajectory. The same split is reused by the visual, temporal, and final-estimator experiments in Chapter 5.

Table 錯誤! 找不到參照來源。 defines the route-level split used throughout the thesis. The Split column separates model-development and held-out partitions, Route identifies the flight trajectory, Frames gives the number of retained images after filtering, and Role states how each subset is used. Because this split is fixed once at the route level, all later ablations and co

Comparisons use the same training and held-out frames rather than regenerating a different partition for each experiment.

Table 1. FieldAnchor route statistics after flight-height filtering. Route A is the only training route. Routes B and C are reserved for held-out evaluation; their 3,534 frames form the main B+C test set used for aggregate reporting in Chapter 5.

Split	Route	Frames	Role
Train	A	1,732	Visual-model training
Test	B	2,276	Held-out evaluation
Test	C	1,258	Held-out evaluation
Total	A+B+C	5,266	Registered real-flight frames

As shown in Table 錯誤! 找不到參照來源。 , Route A contains 1,732 training frames, while Routes B and C contain 2,276 and 1,258 held-out frames, respectively. The two held-out routes therefore provide 3,534 evaluation frames, and the complete registered dataset contains 5,266 retained frames. This fixed route-level allocation is reused throughout Chapter 5, so changes in experimental results cannot be attributed to a changing train/test split.

3.2 Fixed Satellite Anchor Lattice

The satellite orthomosaic is partitioned once into a permanent anchor lattice. Each anchor has a stable identifier, grid row and column, 320×320 pixel crop, metric center, and cached frozen backbone feature. The lattice stride is 32 pixels, providing a dense spatial response field while allowing every UAV frame to query the same database. The released lattice contains 95,480 anchors. With the archived map registration, one 32-pixel gallery step corresponds to approximately one local lattice cell (about 4.75 m in the current experiment configuration), which provides the metric sampling scale used when constructing the forward local candidate field.

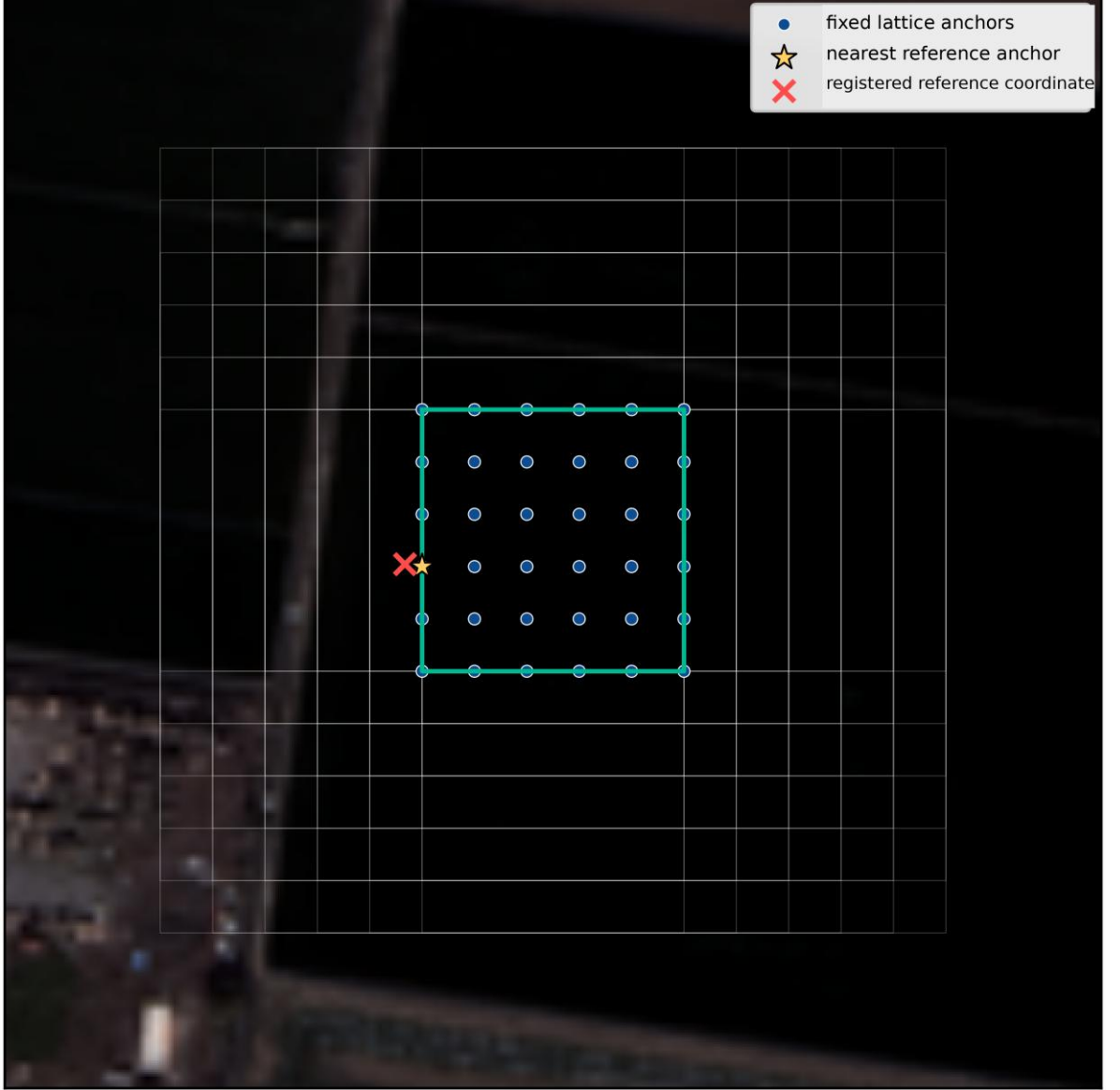


Figure 2. Permanent satellite lattice and a controlled local candidate window. The permanent satellite anchor lattice is fixed globally, while each query opens only one controlled local candidate window around the supplied coarse position estimate. The lattice provides stable anchor identities, metric centers, and cached visual features; the per-frame window only selects a compact subset from that permanent gallery. This separation keeps the candidate database identical across methods and evaluation runs.

3.3 Controlled Local Candidate Construction

A local-candidate manifest is generated before model training and evaluation so that every method receives an identical geometric search condition. Each retained UAV observation is associated with a predefined route reference point on the known planned trajectory. For the controlled local-search benchmark, a coarse local position estimate is formed by applying the documented deterministic spatial perturbation to the associated reference point. This makes the local search center intentionally imperfect while keeping the candidate geometry repeatable across methods. The predefined route reference point is stored separately for supervision and eval

uation. Given the coarse estimate, the nearest permanent satellite anchor is found and one contiguous 6×6 local lattice window is recorded with stable anchor identifiers, metric centers, and cached descriptors. Neither the coarse estimate nor the predefined route reference point is concatenated with the UAV or satellite visual embedding.

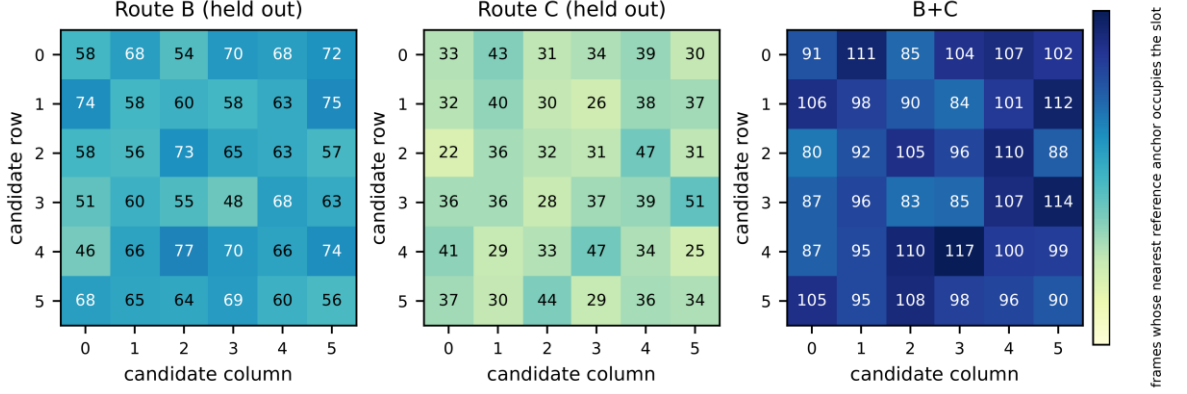


Figure 3. Reference-anchor slot distributions in the local candidate grid. The reference-anchor slot distribution spans the local grid rather than concentrating at one fixed array location. This verifies that the controlled local-search task cannot be solved by memorizing a constant candidate index: the localizer must still use visual evidence together with the candidate geometry to identify the correct region.

Algorithm 1 consolidates the controlled local candidate construction into two explicit stages. Step 1 maps the supplied coarse local position estimate to the nearest anchor in the permanent satellite lattice. Step 2 then constructs the repeatable 6×6 neighborhood and retrieves the stable anchor identifier, metric center, and cached satellite descriptor for every candidate. The resulting object is the complete 36-candidate geometric neighborhood; the hard forward 3×6 restriction is applied later in Section 4.3 before online satellite projection and similarity scoring. In this way, the coarse estimate determines only where the local gallery is opened and is not used as a visual feature or reported as the localization output.

For Algorithm 1, GridIndex denotes the deterministic conversion from the nearest permanent anchor to its lattice row and column indices, and RegularGrid denotes construction of the fixed local candidate window centered on those indices. These are geometric indexing operations with no learned parameters.

Algorithm 1: Controlled Local Candidate Construction

Input: coarse local position estimate $\mathcal{C}_t^{coarse}$, fixed satellite lattice G , and lattice stride;
Output: ordered local candidate set $C_t^{6 \times 6}$ with metric centers and cached descriptors;

// Step 1: Locate the coarse-position anchor in the permanent lattice

- 1 $i_0 \leftarrow \operatorname{argmin}_i \|c_i - \mathcal{C}_t^{coarse}\|_2$;
- 2 $(row_0, col_0) \leftarrow \text{GridIndex}(i_0)$;

// Step 2: Construct one repeatable local candidate window

- 3 $C_t^{6 \times 6} \leftarrow \text{RegularGrid}(row_0, col_0, 6, 6, stride)$;
- 4 **for each** $i \in C_t^{6 \times 6}$ **do**
- 5 read anchor ID i , metric center c_i , and cached satellite descriptor f_i^s ;
- 6 **end for**
- 7 **return** $C_t^{6 \times 6}$ and c_i, f_i^s ;

Step 1 of Algorithm 1 corresponds to Lines 1–2: the nearest permanent anchor is located in metric space and converted to its lattice row and column. Step 2 corresponds to Lines 3–6: the regular local grid is instantiated around that anchor and the candidate metadata needed by the later visual localizer are read without recomputing the frozen satellite backbone. Line 7 returns the complete local candidate object used by the forward-selection stage in Chapter 4.

Together, Figure 3 and Algorithm 1 describe complementary aspects of the same benchmark operation. The figure verifies that the reference region can occur at different slots within the local grid, while the algorithm specifies how that grid is reproduced deterministically for every frame. This establishes a fixed geometric input condition before any learned similarity scoring is performed.

3.4 Scope and Data Release

FieldAnchor provides the image, map-registration, route-split, anchor-lattice, planned-route reference-point, and local-candidate objects required for reproducible visual relocalization. Chapter 4 extends these dataset objects with the FieldAnchor-LR temporal model, inertial prediction, uncertainty estimation, and Kalman filtering. This separation keeps the dataset protocol reusable while allowing the localization model to evolve independently.

The dataset package and the final FieldAnchor-LR model share the same route split, map registration, satellite lattice, planned-route geometry, controlled coarse-position condition, and predefined route reference points. The coarse-position condition defines only the candidate neighborhood, whereas the reference points are used for supervised targets and evaluation metrics. Model-specific temporal states are introduced only in Chapter 4 and are not stored as visual input annotations.

3.5 Acquisition Filtering and Image Preprocessing

The benchmark preparation process separates geometric information from visual model inputs. The retained frame list is first determined from the registered flight sequence and the documented quality filter. Each retained UAV image is center-cropped to 256×256 pixels without resizing the satellite crop. This gives a stable input field of view across training and testing, and prevents a change of image scale from being confounded with a change of candidate geometry.

Satellite patches are cropped directly from the registered orthomosaic. The 320×320 pixel patch size is intentionally larger than the 32-pixel lattice stride, so adjacent anchors have substantial image overlap. The overlap is not treated as independent additional imagery. It creates a locally continuous spatial response field in which a correct visual region should receive support from nearby anchors.

The same preprocessing is used in training and held-out evaluation: retained UAV frames are center-cropped to 256×256 pixels, satellite candidates are cropped directly from the orthomosaic at 320×320 pixels, and the fixed lattice uses a 32-pixel stride. Registration coordinates are never concatenated with the visual feature vector.

The archived frame list, crop origin, resize rule, and lattice metadata are released with the protocol so that evaluation does not silently change the visual evidence associated with an anchor.

3.6 Map Registration and Metric Coordinate System

FieldAnchor associates each retained UAV frame and satellite anchor with a common local metric coordinate system. Image-grid coordinates are useful for extracting satellite crops, but all localization errors and mean-shift kernels are computed in meters. This prevents the kernel bandwidth from depending on orthomosaic pixel resolution or image export settings.

Let $\tilde{u}_i = [u_i, v_i, 1]^\top$ denote the homogeneous pixel coordinate of satellite anchor i , let $c_i \in R^2$ denote its local metric center, and let Π denote the fixed pixel-to-metric registration matrix. The stored metric center is defined by

$$c_i = \Pi \tilde{u}_i, \quad \tilde{u}_i = [u_i, v_i, 1]^\top. \quad (1)$$

The matrix Π is fixed for the archived orthomosaic derivative. The operational model does not estimate Π ; it only uses the metric centers already stored in the anchor lattice. By isolating registration from visual similarity learning, the evaluation can distinguish map geometry errors from response-ranking errors.

All metric computations occur after registration. The model sees satellite pixels and cached visual descriptors, while the decoder reads only the candidate centers associated with those descriptors.

This representation also clarifies the scope of the reported error. Mean Localization Error (MLE) measures the Euclidean distance between the predicted metric position and the corresponding predefined route reference coordinate. It is a visual-localization error measure rather than a GPS receiver-accuracy measure.

3.7 Benchmark Package, Privacy Boundary, and Versioning

A reproducible local-localization benchmark requires more than image folders. The essential objects are the fixed lattice, route split files, candidate manifests, feature-cache schema, model configuration, and metric implementation. Together they ensure that a future model is evaluated on the same 36 local candidates for each frame rather than on a newly generated neighborhood with different coverage.

The package layout is designed so that the response decoder can be reproduced without exposing raw flight logs or personal source-machine paths.

Raw videos, sensor logs, timestamps, yaw values, altitude records, source-machine paths, and unredacted geographic metadata are outside the visual-anchor experiment. Keeping them separate limits avoidable privacy and location leakage; any public release should document permissions, orthomosaic provenance, redaction, licensing, and dataset version hashes.

Chapter 4 Proposed Method

4.1 Overview

Let t denote the current UAV frame index. The system receives the current UAV image and the two preceding UAV images, a georeferenced satellite orthomosaic, the known planned route, and a controlled coarse local position estimate c_t^{coarse} for the current query. The coarse estimate defines only the local satellite search neighborhood. The visual encoder and recurrent representation remain free of explicit map coordinates, while the final estimator operates in continuous route coordinates and returns a two-dimensional metric map position.

The FieldAnchor Local Relocalization (FieldAnchor-LR) framework contains six stages. A frozen MobileCLIP2-S2 encoder and two trainable view-specific projectors first construct cross-view embeddings. A complete 6×6 local satellite geometry is formed. Before online satellite projection and similarity scoring, a hard candidate restriction retains only the forward 3×6 subset; the rear half is excluded. Only the retained candidates are scored. Mean-Shift (MS) converts the visual similarities into a continuous visual observation and response uncertainty. A three-frame Gated Recurrent Unit (GRU) then estimates motion and measurement information from recent visual features and the previous state. The estimated velocity and acceleration form a second-order inertial polynomial. Finally, an external uncertainty-aware Kalman Filter (KF) fuses the motion prediction with the MS visual measurement in continuous route coordinates.

The term “Hard” refers to the explicit forward candidate restriction applied before Mean-Shift decoding: only the route-aligned forward 3×6 subset is retained from the complete 6×6 local candidate geometry. It does not denote a separate variant of the Mean-Shift algorithm. Accordingly, “Hard” is used only for this candidate-support decision, whereas the spatial decoder itself is referred to uniformly as Mean-Shift (MS) throughout the thesis.

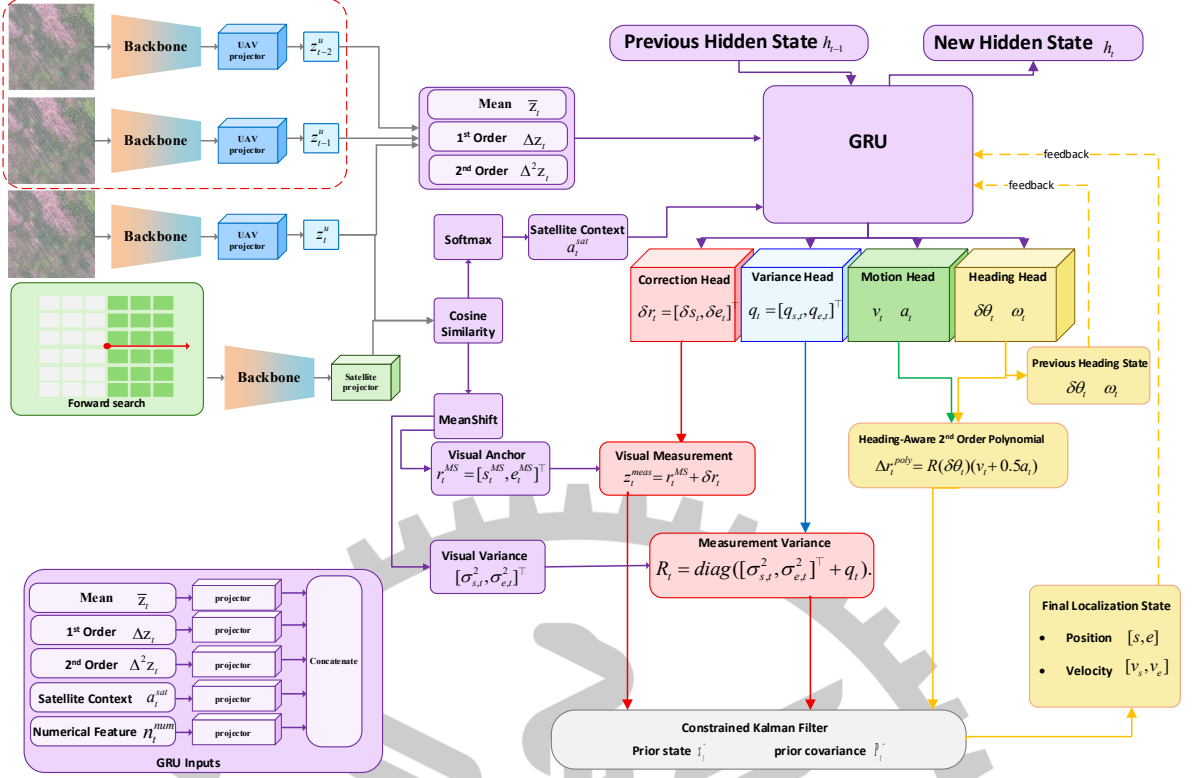


Figure 4. Overview of the FieldAnchor-LR local relocation framework. The processing sequence is read from local-gallery construction to final state estimation. The coarse local position estimate terminates at candidate construction and is not appended to the UAV embedding, satellite embedding, or recurrent feature vector. The complete 6×6 geometry is first reduced by the hard forward 3×6 candidate restriction; visual matching is then performed only on the retained 18 candidates. Mean-Shift converts the resulting response field into a continuous observation, the GRU estimates temporal motion and measurement information, and the external Kalman Filter fuses the motion prediction with the visual measurement to produce the final continuous route state.

Each major block therefore has a single handoff to the next stage: visual representation produces candidate response scores, spatial decoding produces a metric observation and uncertainty, recurrent modeling produces motion and correction terms, and filtering produces the final route state. Presenting the method in this sequence keeps the learned visual representation, learned temporal state, and explicit estimator conceptually separate while still allowing them to operate as one causal localization pipeline.

4.2 UAV–Satellite Visual Representation

Before defining the similarity score, let $E(\cdot)$ denote the frozen MobileCLIP2-S2 image encoder [12], let $P_u(\cdot)$ and $P_s(\cdot)$ denote the trainable UAV and satellite projection heads, respectively, let I_t^u denote the current UAV image, and let $I_{t,i}^s$ denote satellite candidate i . The corresponding L_2 -normalized 512-dimensional embeddings z_t^u and $z_{t,i}^s$ are defined as

$$z_t^u = \frac{P_u(E(I_t^u))}{\|P_u(E(I_t^u))\|_2}, \quad z_{t,i}^s = \frac{P_s(E(I_{t,i}^s))}{\|P_s(E(I_{t,i}^s))\|_2}, \quad (2)$$

where the frozen encoder supplies a shared pretrained representation and the two independent projection heads adapt that representation to UAV-view and satellite-view appearance. Satellite backbone descriptors are computed once and cached, whereas the lightweight satellite projector is evaluated only for the retained online candidates.

Let α denote the learnable logit-scale parameter. The cosine-similarity logit $l_{t,i}$ of candidate i is

$$l_{t,i} = \exp(\alpha)(z_t^u)^\top z_{t,i}^s. \quad (3)$$

Before using the candidate posterior, let $\tau > 0$ denote the response temperature and let N denote the number of retained candidates. The normalized visual posterior $p_{t,i}$ is

$$p_{t,i} = \frac{\exp(\frac{l_{t,i}}{\tau})}{\sum_{j=1}^N \exp(\frac{l_{t,j}}{\tau})}, \quad \sum_{j=1}^N (p_{t,j}) = 1, \quad (4)$$

where the posterior sums to one over the retained local candidates. FieldAnchor-LR uses $\tau=0.30$. The same posterior supplies the response mass for MS and the satellite context used by the temporal model. In Algorithm 2, this normalization is the first operation of Step 1, after which each retained candidate center is initialized as an independent Mean-Shift seed.

4.3 Hard Forward 3×6 Candidate Restriction

The candidate geometry is first constructed as a complete 6×6 neighborhood containing 36 candidates around the supplied coarse local position estimate. Let $c_t \in \mathbb{R}^2$ denote the neighborhood center, let $c_{t,i} \in \mathbb{R}^2$ denote candidate center i , and let θ_t denote the causal search direction. Define the forward unit vector $f_t = [\cos \theta_t, \sin \theta_t]^\top$ and the perpendicular unit vector $n_t = [-\sin \theta_t, \cos \theta_t]^\top$. The longitudinal and lateral projections are

$$d_{t,i}^\parallel = (c_{t,i} - c_t)^\top f_t, \quad d_{t,i}^\perp = (c_{t,i} - c_t)^\top n_t, \quad (5)$$

where $d_{t,i}^{\parallel}$ measures the forward displacement and $d_{t,i}^{\perp}$ measures the lateral displacement. The 18 candidates with the largest forward projection are retained and ordered from front to back and left to right. This discrete support selection is the “Hard” operation in FieldAnchor-LR: the remaining 18 rear candidates are removed before online satellite projection, cosine-similarity scoring, and Mean-Shift decoding. Only the retained 18 satellite descriptors are projected and scored. The direction is causal because it combines the local route tangent with the previously estimated heading residual and does not use any future frame.

Causal Forward Candidate Restriction

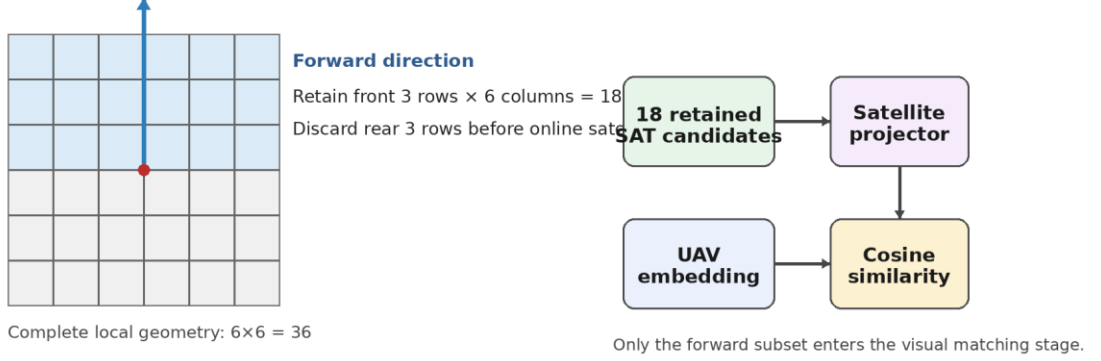


Figure 5. Hard forward 3×6 candidate restriction. The diagram gives the geometric interpretation of Equation (5): the complete 6×6 local neighborhood is projected into route-aligned longitudinal and lateral coordinates, after which the hard restriction retains only the causal forward 3×6 half-grid. These retained 18 candidates form the response field used by the subsequent visual decoder, while candidates behind the current motion direction are excluded from online scoring.

The hard forward restriction is causal: the orientation of the retained half-grid is determined from the planned-route tangent and the previously estimated heading residual, not from a future UAV frame. Consequently, the candidate reduction changes the spatial support and online scoring cost without introducing future-frame information into the localizer.

4.4 Mean-Shift Visual Localization

After the hard forward restriction, $N = 18$ metric candidate centers remain. Mean-Shift (MS) [11] is used as a continuous spatial decoder rather than a winner-take-all anchor selector. Let $m_{t,i}^{(k)} \in \mathbb{R}^2$ denote the mode initialized from candidate i after k updates, let $h > 0$ denote the Gaussian kernel bandwidth, and let K denote the number of iterations. The initialization is $m_{t,i}^{(U)} \in c_{t,i}$. Each mode is updated by

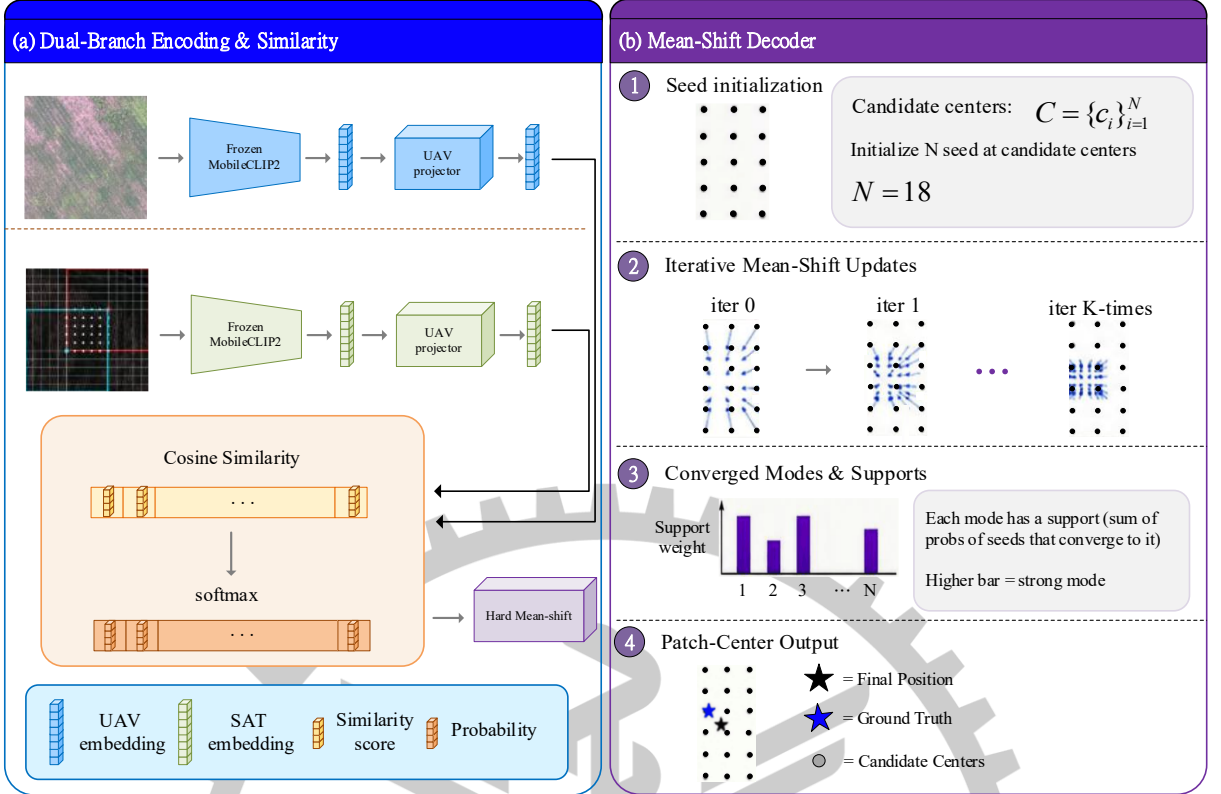


Figure 6. Mean-Shift (MS) continuous visual localization flow. The three visualized operations correspond to Algorithm 2. Candidate logits are normalized and candidate centers initialize the seeds; the seeds are iteratively shifted toward locally supported response mass; and the converged modes are weighted to form the continuous visual coordinate and uncertainty. The figure therefore visualizes the same computation defined algebraically in Equations (6)–(9).

$$m_{t,i}^{(k+1)} = \frac{\sum_{j=1}^N p_{t,j} \exp\left(-\frac{\|m_{t,i}^{(k)} - c_{t,j}\|_2^2}{2h^2}\right) c_{t,j}}{\sum_{j=1}^N p_{t,j} \exp\left(-\frac{\|m_{t,i}^{(k)} - c_{t,j}\|_2^2}{2h^2}\right)}, \quad (6)$$

where $h = 8$ m and $K = 3$ in FieldAnchor-LR. Every candidate seed remains active during the update, allowing nearby candidates with consistent visual support to converge toward the same spatial mode. This iterative update is Step 2 of Algorithm 2 (Lines 3–8); the nested loops make explicit that every seed is repeatedly updated using the response mass from all retained candidates inside the Gaussian kernel.

The denominator normalizes the kernel-weighted posterior mass, whereas the numerator is the corresponding weighted sum of candidate centers. These terms do not cancel because each weight in the numerator is multiplied by a different candidate center before summation. Equation (6) therefore returns a weighted average location for each Mean-Shift seed rather than reducing to a single candidate center.

Let $\rho_{t,i}$ denote the density support of the i -th converged mode. It is defined by

$$\rho_{t,i} = \sum_{j=1}^N p_{t,j} \exp\left(-\frac{\|m_{t,i}^{(k)} - c_{t,j}\|_2^2}{2h^2}\right). \quad (7)$$

Let $\beta > 0$ denote the mode-density temperature and let $\omega_{t,i}$ denote the normalized mode weight. FieldAnchor-LR uses $\beta = 12$, and the weights are

$$\omega_{t,i} = \frac{\exp(\beta \rho_{t,i})}{\sum_{r=1}^N \exp(\beta \rho_{t,r})}, \quad \sum_{i=1}^N \omega_{t,i} = 1. \quad (8)$$

The final continuous MS visual coordinate x_t^{MS} is

$$x_t^{MS} = \sum_{i=1}^N \omega_{t,i} m_{t,i}^{(K)}, \quad (9)$$

so the visual observation is not snapped to a discrete satellite anchor. Equations (7)–(9) constitute Step 3 of Algorithm 2 (Lines 9–13): density support is computed for each converged mode, the supports are converted to normalized mode weights, and the final continuous position is formed as their weighted combination. The same converged mode distribution is retained to estimate visual measurement uncertainty before temporal fusion.

Algorithm 2 consolidates the response normalization in Equation (4) and the continuous Mean-Shift operations in Equations (6)–(9) into one executable sequence. Unlike a discrete Top-1 selector, the procedure keeps all candidate seeds active during mode seeking and returns both the continuous visual coordinate and the mode-space uncertainty required by the later state estimator.

For Algorithm 2, Softmax denotes the normalized exponential operation already defined by Equations (4) and (8). WeightedModeVariance denotes the weighted second-moment calculation that uses the converged modes and normalized mode weights to obtain the mode-space variance described in Section 4.8. These names are shorthand for the stated calculations rather than additional learned modules.

Algorithm 2: Mean-Shift (MS) Visual Localization

Input: candidate logits l_i , metric centers c_i , temperature τ , bandwidth h , iterations K , and density scale β ;

Output: continuous visual coordinate x^{MS} , mode-space variance Σ^{MS} , and mode weights ω_i ;

```
// Step 1: Normalize the visual response and initialize the seeds
1  $p \leftarrow \text{Softmax}(\frac{l}{\tau});$ 
2 for  $i \leftarrow 1$  to  $N$  do  $m_i \leftarrow c_i$ ; end for
// Step 2: Iteratively shift every seed toward local response mass
3 for  $k \leftarrow 0$  to  $K-1$  do
4   for  $i \leftarrow 1$  to  $N$  do
5      $w_{ij} \leftarrow p_j \exp(-\frac{\|m_i - c_j\|_2^2}{2h^2})$  for all  $j$ ;
6      $m_i \leftarrow (\frac{\sum_j (w_{ij})c_j}{\sum_j (w_{ij})});$ 
7   end for
8 end for
// Step 3: Density-weight the converged modes and form the output
9  $\rho_i \leftarrow \sum_j (p_j) \exp(-\frac{\|m_i - c_j\|_2^2}{2h^2});$ 
10  $\omega \leftarrow \text{Softmax}(\beta \rho);$ 
11  $x^{MS} \leftarrow \sum_i (\omega_i)m_i;$ 
12  $\Sigma^{MS} \leftarrow \text{WeightedModeVariance}(m_i, \omega_i);$ 
13 return  $x^{MS}$ ,  $\Sigma^{MS}$ , and  $\omega_i$ ;
```

Reading Algorithm 2 sequentially, Lines 1–2 establish the normalized response field and seed locations, Lines 3–8 perform the iterative local mode search, and Lines 9–13 convert the converged modes into a single visual coordinate and uncertainty estimate. The output x_t^{MS} is projected into the continuous route coordinate system in Section 4.5, whereas Σ_t^{MS} is propagated to the uncertainty model in Section 4.8. Thus the algorithm is the direct bridge between frame-level visual similarity and the temporal estimator.

4.5 Continuous Route Coordinates and Waypoint Representation

The temporal estimator uses continuous route coordinates so that forward progress and lateral displacement retain clear physical meaning. Let q_k and q_{k+1} denote consecutive predefined route reference points, let $L_k = \|q_{k+1} - q_k\|_2$ denote the segment length, let $u_k = \frac{q_{k+1} - q_k}{L_k}$ denote the segment unit vector, let $n_k = [-u_k, y, u_k, x]^\top$ denote the cross-track unit vector, and let

S_k denote the accumulated route length at q_k . For route state $r = [s, e]^\top$ on segment k , the metric coordinate is

$$x(s, e) = q_k + (s - S_k)u_k + en_k, \quad (10)$$

where s is along-route progress and e is signed cross-track displacement. Around a waypoint, an instantaneous switch of the cross-track axis would rotate the same nonzero lateral state discontinuously. FieldAnchor-LR therefore rotates the local route frame causally after the filtered progress passes the waypoint, which avoids premature turning while preserving a continuous XY trajectory. The MS position is projected to this route coordinate before recurrent state estimation.

4.6 Three-Frame Gated Recurrent Unit Temporal Modeling

Let z_t , z_{t-1} , and z_{t-2} denote the current and two preceding UAV embeddings after the view-specific projection head. The three-frame representation uses a temporal mean, the most recent first difference, and the corresponding second difference, defined by

$$\bar{z}_t = \frac{z_{t-2} + z_{t-1} + z_t}{3}, \quad \Delta z_t = z_t - z_{t-1}, \quad \Delta^2 z_t = (z_t - z_{t-1}) - (z_{t-1} - z_{t-2}). \quad (11)$$

The first difference captures the most recent change in visual representation, whereas the second difference describes how that change itself evolves. In the notation used below, the superscript labels “sat”, “vis”, and “num” are descriptive abbreviations for satellite, visual, and numeric, respectively; they identify the role of a quantity and do not denote exponentiation. The satellite context is formed by posterior-weighting the retained satellite embeddings with the normalized candidate posterior from Equation (4). The visual innovation is the route-coordinate difference between the current projected Mean-Shift observation and the one-step causal motion prior available before Kalman fusion. The two mode-space variance components used by the numeric state are produced by the Mean-Shift decoder and are written explicitly in Equation (17). The scalar turn-rate component of the heading state is distinct from the indexed Mean-Shift mode weights defined in Equation (8).

Let a_t^{sat} denote the posterior-weighted satellite context, let δ_t^{vis} denote the innovation between the current MS observation and the motion prediction, let v_{t-1} denote the previous route velocity, and let $h_{t-1}^\theta = [\delta\theta_{t-1}, \omega_{t-1}]^\top$ denote the previous heading residual and turn-rate state. The eight-dimensional numeric state n_t^{num} contains the two log mode-space variances, the two-di

mensional visual innovation, the two-dimensional previous velocity, and the two-dimensional heading state h_{t-1}^θ .

The five projection symbols introduced below denote learned feature-projection blocks applied separately to the temporal mean, first difference, second difference, satellite context, and numeric state. Their role is to place these heterogeneous inputs in compatible feature spaces before fusion; they do not create additional route states or geometric measurements. For Equation (12), the semicolon-separated bracket notation is defined as feature concatenation, meaning that the projected feature blocks are appended along the feature dimension before being passed to the recurrent unit.

Let Φ_μ , Φ_Δ , Φ_Δ^2 , Φ_s , and Φ_n denote learnable projection functions for the temporal mean, first difference, second difference, satellite context, and numeric state. The recurrent input g_t is

$$g_t = [\Phi_\mu(\bar{z}_t); \Phi_\Delta(\Delta z_t); \Phi_\Delta^2(\Delta^2 z_t); \Phi_s(a^t); \Phi_n(n_t^{num})]. \quad (12)$$

The operator $[\cdot; \cdot]$ denotes feature concatenation. Let h_{t-1} denote the previous recurrent hidden vector. The current hidden state is

$$h_t = \text{GRUCell}(g_t, h_{t-1}). \quad (13)$$

The GRU hidden dimension is 256. Four lightweight output heads decode the hidden state into route velocity and acceleration, heading residual and turn rate, visual-measurement correction, and positive extra measurement variance. The heading variables remain internal motion states and are not treated as a separate final localization output.

The GRUCell in Equation (13) follows the standard Gated Recurrent Unit formulation [13]. It computes reset and update gates with sigmoid activation from the current recurrent input and previous hidden vector, forms a candidate hidden representation using a hyperbolic-tangent activation with reset-gated history, and then interpolates between the previous and candidate hidden representations. This recurrent update is causal and uses no future-frame information.

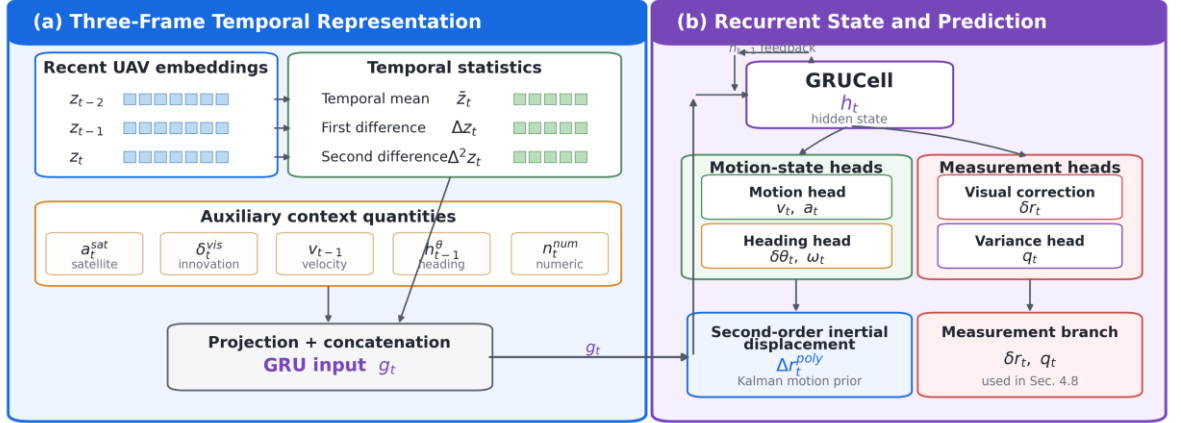


Figure 7. Three-frame GRU and second-order inertial prediction.

The architecture in Figure 7 groups the temporal estimator into two coupled parts. The three recent UAV embeddings first form the temporal mean, first difference, and second difference; these features are combined with the satellite context, visual innovation, previous route velocity, previous heading state, and the numeric state to construct the GRU input. The recurrent hidden state is then decoded by separate motion, heading, visual-correction, and variance heads. Only the motion and heading outputs form the second-order inertial displacement, whereas the visual-correction and variance outputs are passed to the measurement branch described in Section 4.8.

This arrangement also explains why three frames are used even though the final output is one position per frame. The current observation supplies the present visual evidence, while the two preceding embeddings make both a recent change and a change-of-change observable to the GRU. The temporal ablation in Table 錯誤! 找不到參照來源。 later evaluates whether the additional history improves held-out localization.

4.7 Second-Order Inertial Motion Prediction

The recurrent motion head produces route velocity $v_t = [v_t^\parallel, v_t^\perp]^\top$ and route acceleration $a_t = [a_t^\parallel, a_t^\perp]^\top$. Let $\delta\theta_t$ denote the causal heading residual, and let $R(\delta\theta_t)$ denote the corresponding two-dimensional rotation matrix

$$R(\delta\theta_t) = [\cos \delta\theta_t, -\sin \delta\theta_t; \sin \delta\theta_t, \cos \delta\theta_t], \quad (14)$$

where $\delta\theta_t$ is estimated causally from the recurrent state together with the preceding temporal state. With one frame treated as one discrete time step, the second-order inertial displacement is

$$\Delta r_t^{poly} = R(\delta\theta_t)(v_t + \frac{1}{2}a_t). \quad (15)$$

With one frame treated as one discrete time step, Equation (15) combines the GRU-predicted velocity and acceleration through the second-order polynomial and rotates the resulting displacement by the predicted heading residual. The resulting displacement is passed directly to the Kalman prediction as the learned motion prior for the current frame.

4.8 Visual Measurement Correction and Uncertainty Estimation

Let $r_t^{MS} = [s_t^{MS}, e_t^{MS}]^\top$ denote the route-coordinate projection of the continuous MS observation, and let $\delta r_t = [\delta s_t, \delta e_t]^\top$ denote the correction predicted by the GRU. The visual measurement supplied to the KF is

$$z_t^{meas} = r_t^{MS} + \delta r_t, \quad (16)$$

To quantify the visual response itself, let u_t and n_t denote the local route-parallel and cross-track unit vectors, let $m_{t,i}^{(K)}$ denote the converged MS modes, and let $\omega_{t,i}$ denote the mode weights. The mode-space variances are

$$\sigma_{s,t}^2 = \sum_i \omega_{t,i} [(m_{t,i}^{(K)} - x_t^{MS})^\top u_t]^2, \quad \sigma_{e,t}^2 = \sum_i \omega_{t,i} [(m_{t,i}^{(K)} - x_t^{MS})^\top n_t]^2, \quad (17)$$

where tightly converged modes yield a small variance and spatially separated plausible modes increase the uncertainty. Let $q_t = [q_{s,t}, q_{e,t}]$ denote the non-negative extra variance predicted by the GRU through a softplus output. The measurement covariance is

$$R_t = \text{diag}([\sigma_{s,t}^2, \sigma_{e,t}^2]^\top + q_t). \quad (18)$$

In Equation (18), softplus denotes the standard smooth positive mapping used to convert the variance-head output into the non-negative extra variance vector q_t . The function $\text{diag}()$ is the diagonal-matrix operator, which places the elements of its vector argument on the main diagonal. Thus, the measurement covariance is formed directly from the Mean-Shift mode-space variance and the learned extra variance.

4.9 Uncertainty-Aware Kalman Filtering

The final estimator operates in route coordinates. Let $x_t = [s_t, e_t, v_{s,t}, v_{e,t}]^\top$ denote the KF state, let P_t denote its covariance, let F denote the constant-velocity transition matrix, let Q denote the process-noise covariance, let H denote the 2×4 position-observation matrix, and let R_t denote the measurement covariance defined above. The recurrent polynomial displacement is injected into the position prediction according to Equation (19). The four state components

ts are ordered as along-route progress, cross-track displacement, along-route velocity, and cross-track velocity.

$$x_t^- = Fx_{t-1} + b(\Delta r_t^{poly}), \quad P_t^- = FP_{t-1}F^\top + Q, \quad (19)$$

The injection function $b(\cdot)$ inserts the two-dimensional polynomial displacement into the position components of the four-dimensional state. Let v_t denote the visual innovation, let S_t denote the innovation covariance, and let η_t denote the Normalized Innovation Squared (NIS) statistic. They are

$$v_t = z_t^{meas} - Hx_t^-, \quad S_t = HP_t^-H^\top + R_t, \quad \eta_t = v_t^\top S_t^{-1}v_t, \quad (20)$$

where a large NIS indicates disagreement between the inertial prediction and the current visual measurement. The NIS is retained as a consistency statistic describing agreement between the inertial prediction and the current visual measurement, while the visual reliability entering the Kalman gain is represented by the measurement covariance defined in Equation (18). Let K_t denote the Kalman gain and let I denote the 4×4 identity matrix. The covariance-stable update is

$$K_t = P_t^- H^\top (HP_t^- H^\top + R_t)^{-1}, \quad x_t = x_t^- + K_t v_t, \quad P_t = (I - K_t H) P_t^- (I - K_t H)^\top + K_t R_t K_t^\top. \quad (21)$$

Equation (21) gives the posterior state and covariance used by the next frame. Route progress and cross-track displacement remain components of the same continuous route-coordinate state, while their updates are determined by the motion prediction, the visual measurement, and the learned measurement covariance through the Kalman gain.

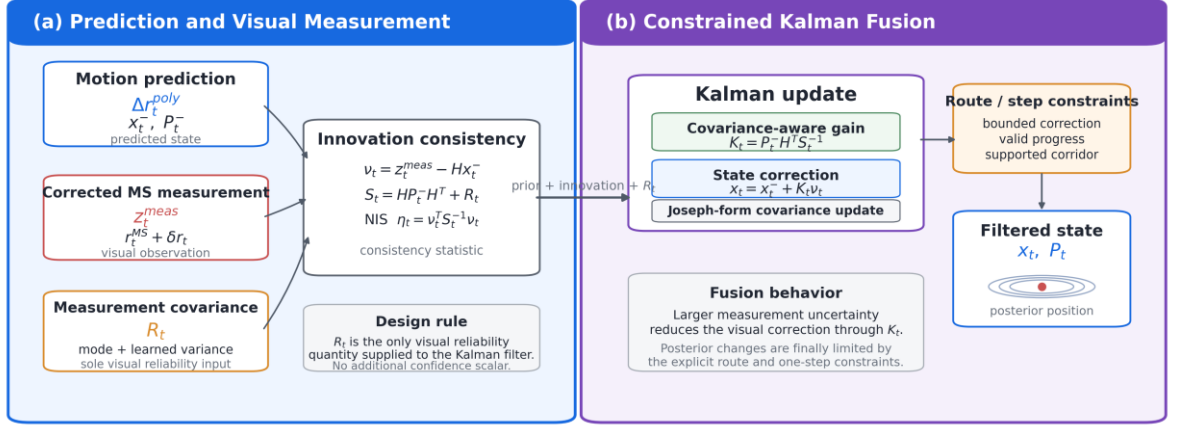


Figure 8. Uncertainty-aware Kalman fusion. Figure 8 separates the Kalman stage into prediction and measurement formation followed by uncertainty-aware fusion. The motion model supplies the prior state and covariance, the corrected Mean-Shift observation supplies the position measurement, and the measurement covariance R_t supplies the visual uncertainty used by the Kalman gain. Their disagreement is summarized by the innovation and NIS, after which the covariance-aware update produces the posterior state and covariance.

The diagram emphasizes that R_t is the measurement covariance and the sole visual reliability quantity supplied to the Kalman Filter. It enters the covariance-aware update directly, while the recurrent motion state remains a separate source of temporal prediction; no additional reliability input is supplied to the filter.

4.10 Training Objective

Training is divided into a visual stage and a temporal stage. In the visual stage, only Route A is used to optimize the two projection heads while the MobileCLIP2-S2 backbone remains frozen. Each training observation first retrieves the satellite candidates associated with its controlled coarse local position estimate. The candidate nearest the corresponding predefined route reference point provides the classification target, while the continuous MS coordinate is supervised directly against that reference point. Let L_{CE} denote cross-entropy over the local satellite candidates, let L_{coord} denote the Smooth L1 coordinate loss, and let L_{visual} denote the visual-stage objective. The objective is

$$L_{visual} = L_{CE} + 0.25L_{coord}, \quad (22)$$

where label smoothing is applied to L_{CE} . The temporal stage is trained sequentially with Truncated Backpropagation Through Time (TBPTT) over contiguous Route-A frames under the same controlled local-search condition. Consecutive predefined route reference points provide the supervised motion targets, while the GRU itself receives recent visual features, satellite context, and previous model state. Let L_{meas} denote the Smooth L1 visual-measurement loss, L_{step} the Smooth L1 next-step displacement loss, L_{vel} a weak velocity loss, L_{head} the periodic heading loss, and L_{var} the Gaussian negative log-likelihood for measurement variance. The active temporal objective is

$$L_{temporal} = 1.00L_{meas} + 3.00L_{step} + 0.25L_{vel} + 1.25L_{head} + 0.05L_{var}. \quad (23)$$

Acceleration is not assigned an independent supervised term; it is learned through L_{step} because it directly contributes to the second-order displacement. Turn rate, speed, route progress, and a separate cross-motion regularizer are not active independent losses in FieldAnchor-LR. At each TBPTT boundary the computational graph is detached, while the recurrent and filter states themselves remain continuous. The predefined route reference points used to form the losses are training annotations and are not concatenated with the visual embeddings or recurrent input vector. Algorithm 3 organizes these objectives into the same two stages described in the text: Lines 1–9 implement visual representation learning with Equation (22), and Lines 10–19 implement sequential temporal learning with Equation (23).

In Algorithm 3, LocalCandidates refers to Algorithm 1; ViewProjectors and ScaledCosine refer to the operations in Equations (2)–(4); MeanShift refers to Algorithm 2; ForwardCandidates refers to the hard forward 3×6 candidate restriction defined by Equation (5); VisualObservation denotes the continuous Mean-Shift measurement described in Sections 4.4 and 4.8; GRU denotes the recurrent computation in Equations (12)–(13); and KalmanFilter denotes the prediction and update defined in Equations (19)–(21). The function-style names are therefore compact references to operations already defined in the text, not unspecified black-box modules.

Algorithm 3: FieldAnchor-LR Two-Stage Training Procedure

Input: Route-A UAV frames, predefined Route-A reference points X_{ref} , coarse-position candidate manifests M , fixed satellite gallery G , and frozen MobileCLIP2-S2;
Output: trained UAV/satellite projectors and three-frame GRU parameters;

// Stage 1: visual representation learning
1 Freeze MobileCLIP2-S2 and cache satellite backbone descriptors;
2 **for each** Route-A minibatch **do**
3 $C \leftarrow \text{LocalCandidates}(\text{coarse position estimate from } M, \text{fixed satellite lattice});$
4 $(z^u, z^s) \leftarrow \text{ViewProjectors}(C, \text{UAV image});$
5 $l \leftarrow \text{ScaledCosine}(z^u, z^s);$
6 $(x^{MS}, \sum^{MS}) \leftarrow \text{MeanShift}(l, \text{candidate centers});$
7 $L_{visual} \leftarrow L_{CE}(\text{nearest reference anchor}) + 0.25 L_{coord}(x^{MS}, x_{ref});$
8 Update UAV and satellite projection heads with AdamW;
9 **end for**
// Stage 2: sequential temporal learning
10 Freeze the visual localizer; initialize GRU hidden state and route-coordinate KF state;
11 **for** $t \leftarrow \text{first}$ **to last contiguous Route-A frame** **do**
12 $C_t \leftarrow \text{ForwardCandidates}(\text{coarse position estimate from } M, \text{previous causal direction});$
13 $(x_t^{MS}, \sum^{MS}, a_t^{sat}) \leftarrow \text{VisualObservation}(C_t);$
14 $(v_t, a_t, \delta\theta_t, \delta r_t, q_t, h_t) \leftarrow \text{GRU}(\text{three-frame features}, \text{previous state});$
15 $\Delta r_t^{poly} \leftarrow R(\delta\theta_t)(v_t + 0.5a_t);$
16 $x_t \leftarrow \text{KalmanFilter}(\Delta r_t^{poly}, r_t^{MS} + \delta r_t, R_t);$
17 accumulate $L_{temporal}$;
18 Every 32 frames: back-propagate, update GRU, and detach graph history;
19 **end for**
20 **return trained projectors and GRU parameters;**

Within the first stage of Algorithm 3, the frozen MobileCLIP2-S2 backbone and cached satellite features are reused while only the view-specific projectors are optimized. Line 6 invokes the Mean-Shift visual localizer summarized in Algorithm 2, and Line 7 applies the visual objective in Equation (22). The second stage then freezes this visual localizer and trains the three-frame GRU sequentially, preserving the recurrent and filter states across contiguous frames while detaching only the graph history at the TBPTT boundary.

The two-stage structure prevents the temporal model from changing the underlying satellite-response representation while it learns motion and measurement behavior. It also makes the training procedure consistent with the inference pipeline: the visual stage produces the same MS observation used online, and the temporal stage learns how that observation interacts with the recurrent motion state and the external uncertainty-aware Kalman Filter.

4.11 End-to-End Inference

During held-out evaluation, UAV frames are processed sequentially with fixed model parameters. For each query, the controlled benchmark supplies a coarse local position estimate that establishes one complete 6×6 satellite neighborhood. The hard forward candidate restriction then retains only the route-aligned forward 3×6 subset, and the current UAV image is matched only against those 18 candidates. The previous two UAV embeddings, recurrent hidden state, motion state, and KF posterior are propagated from the preceding frame and are used for temporal prediction and fusion; they do not replace the benchmark-defined candidate neighborhood. Predefined route reference points are read only after prediction to compute localization metrics. The full held-out localization sequence is summarized in Algorithm 4, which connects hard forward local visual matching, Mean-Shift measurement formation, GRU-based motion estimation, inertial prediction, and uncertainty-aware Kalman fusion.

In Algorithm 4, RouteTangent denotes the planned-route tangent adjusted by the previous causal heading residual; Forward denotes the hard forward candidate restriction in Equation (5); UAVProjector and SatelliteProjector denote the view-specific projections in Equation (2); ScaledCosine denotes Equations (3)–(4); MeanShift denotes Algorithm 2; ProjectToRoute denotes the coordinate projection in Section 4.5; GRU denotes Equations (12)–(13); Measurement Covariance denotes Equations (17)–(18); KFPredict denotes Equation (19); KFUpdate denotes Equations (20)–(21); and RouteToMetricXY denotes conversion from route coordinates to metric map coordinates using Equation (10).

Algorithm 4: FieldAnchor-LR Controlled Local Temporal Localization

Input: current UAV image I_t^u , two recent UAV embeddings, coarse local position estimate c_t^{coarse} , known planned route Q , cached satellite gallery, and previous GRU/KF states;
Output: final metric position p_t and updated recurrent/filter states;

// Hard forward local visual localization

- 1 $c_t \leftarrow c_t^{coarse}$;
- 2 $\theta_t \leftarrow \text{RouteTangent}(\text{previous filtered progress}) + \text{previous heading residual}$;
- 3 $C_t \leftarrow \text{Forward } 3 \times 6(c_t, \theta_{t, \text{fixed}}, \text{satellite lattice})$;
- 4 $z_t^u \leftarrow \text{UAVProjector}(E(I_t^u))$;
- 5 $z_t^s \leftarrow \text{SatelliteProjector}(\text{cached descriptors of } C_t)$;
- 6 $l_t \leftarrow \text{ScaledCosine}(z_t^u, z_t^s)$;
- // Continuous visual measurement and recurrent motion
- 7 $(x_t^{MS}, \sum_t^{MS}) \leftarrow \text{MeanShift}(l_t, \text{centers}(C_t))$;
- 8 $r_t^{MS} \leftarrow \text{ProjectToRoute}(x_t^{MS})$;
- 9 $a_t^{sat} \leftarrow \sum_i (p_{t,i}) z_{t,i}^s$;
- 10 $(v_t, a_t, \delta\theta_t, \delta r_t, q_t, h_t) \leftarrow \text{GRU}(z_t, z_{t-1}, z_{t-2}, a_t^{sat}, \text{previous state})$;
- 11 $\Delta r_t^{poly} \leftarrow R(\delta\theta_t)(v_t + 0.5a_t)$;
- 12 $z_t^{meas} \leftarrow r_t^{MS} + \delta r_t$;
- // Uncertainty-aware state fusion
- 13 $R_t \leftarrow \text{MeasurementCovariance}(\sum_t^{MS}, q_t)$;
- 14 $x_t^- \leftarrow \text{KFPredict}(\text{previous state}, \Delta r_t^{poly})$;
- 15 $x_t \leftarrow \text{KFUpdate}(x_t^-, z_t^{meas}, R_t)$;
- 16 $p_t \leftarrow \text{RouteToMetricXY}(x_t)$;
- 17 Update recent embeddings, recurrent state, and KF state;
- 18 return p_t and updated states;

Algorithm 4 is the operational counterpart of Sections 4.2–4.9. Lines 1–6 correspond to local candidate construction, hard forward restriction, visual projection, and similarity scoring; Lines 7–12 convert the response into an MS observation, project it to route coordinates, and obtain the GRU motion and measurement terms; and Lines 13–18 form the measurement covariance, execute the Kalman predict/update cycle, convert the posterior route state to metric coordinates, update the causal temporal state, and return the final metric position.

This step-by-step correspondence is intentional: Figure 4 gives the global block diagram, Figures 5–8 explain the geometry and state flow inside the major blocks, Equations (2)–(21) define the numerical operations, and Algorithm 4 records their causal execution order for one held-out frame. These elements therefore describe different levels of the same pipeline rather than independent procedures.

4.12 Computational Complexity and Reproducibility

For N retained candidates and K Mean-Shift iterations, each active seed evaluates a kernel against all retained candidates at every iteration, giving a time complexity of $O(KN^2)$. In FieldAnchor-LR, $N = 18$ and $K = 3$, so the Mean-Shift decoder performs $3 \times 18^2 = 972$ pairwise kernel evaluations per frame.

Using the complete 6×6 neighborhood under the same three iterations would require $3 \times 36^2 = 3,888$ pairwise evaluations. Equivalently, the forward 3×6 restriction reduces the Mean-Shift kernel workload by 75% and also halves the number of online satellite projections and cosine-similarity evaluations from 36 candidates to 18.

The repository stores the visual and temporal checkpoints separately, caches the frozen satellite backbone descriptors, and records route-wise prediction files and experiment summaries. Quantitative values are reported only when they are present in versioned outputs; unexecuted settings are marked with an em dash rather than being estimated.

Chapter 5 Experiments and Results

5.1 Experimental Settings and Metrics

All experiments retain the FieldAnchor route split: Route A is used for visual and temporal model training, while Routes B and C are held out for final evaluation. For every query, the same controlled coarse-position condition and fixed satellite lattice are used to establish the complete 6×6 local candidate neighborhood. The predefined route reference point remains separate and is used only to form supervised targets on Route A and to compute metrics on the held-out routes. All methods in a given internal comparison use the same route split, satellite gallery, candidate construction rule, and metric coordinate system. Accordingly, the reported FieldAnchor-LR results quantify local visual refinement around predefined planned-route reference points rather than unrestricted map-wide acquisition.

The primary localization metrics are Mean Localization Error (MLE), Median Localization Error (MedLE), the 90th and 95th percentile localization errors (P90 and P95), and Localization Success Rate within radius r (LSR@ r). Temporal experiments additionally report along-route Progress Mean Absolute Error (Progress MAE), Speed Mean Absolute Error (Speed MAE), abnormal jump statistics when available, end-to-end latency, and Frames Per Second (FP

S) when runtime throughput is reported. Lower MLE, MedLE, P90, P95, Progress MAE, and latency are better; higher LSR is better.

5.2 MS and Weighted-Centroid Accuracy–Runtime Trade-off

This experiment isolates the spatial decoder by comparing probability-weighted centroid decoding with the final Mean-Shift (MS) aggregation under identical candidate responses. Table 2 summarizes both localization accuracy and the isolated coordinate-aggregation cost. In Table 2, Avg. aggregated coordinates indicates how many candidate coordinates contribute to the final decoded point on average; MLE and P90 measure mean and tail error, LSR@ r gives the percentage of frames localized within r meters, and Aggregation time measures only the final coordinate-combination step. The timing therefore excludes image preprocessing, backbone encoding, visual matching, iterative MS updates, GRU inference, and Kalman fusion.

Table 2. Accuracy and coordinate-aggregation runtime comparison between weighted centroid and MS. Weighted Centroid averages all 18 retained candidate centers using the normalized response weights. MS instead performs local mode seeking and combines the converged modes; Avg. aggregated coordinates summarizes how concentrated the final spatial aggregation becomes after decoding.

Decoder	Avg. aggregated coordinates	MLE (m)	P90 (m)	LSR@3 (%)	LSR@5 (%)	LSR@10 (%)	LSR@15 (%)	Aggregation time (ms)
Weighted Centroid	18	3.546	6.953	47.963	77.646	98.189	99.377	0.02755
Mean-Shift (MS)	1.04	3.482	6.620	49.208	76.995	98.359	99.293	0.02141

As shown in Table 錯誤! 找不到參照來源。, MS reduces MLE from 3.546 m to 3.482 m and P90 from 6.953 m to 6.620 m, while increasing LSR@3 from 47.963% to 49.208%. Weighted Centroid is slightly higher at LSR@5 and LSR@15, so the decoder change should be interpreted as improving the mean and tail of the error distribution rather than producing a uniform gain at every threshold. Because the microbenchmark excludes the iterative MS update itself, the aggregation-time column alone is not a complete speed–accuracy comparison; Table 錯誤! 找不到參照來源。 provides the corresponding end-to-end online latency comparison.

5.3 FieldAnchor-LR Component Ablation

This ablation evaluates the contribution of the major FieldAnchor-LR components using the same held-out Routes B and C. Table 3 adds the components in the same order as the proposed pipeline: MS only is the frame-level continuous visual measurement; MS + 3-frame GRU adds temporal representation; MS + GRU + inertial polynomial adds explicit velocity–acceleration propagation; and the final row adds the learned-variance Kalman Filter. MLE, P90, and LSR measure localization accuracy, while Progress MAE measures the along-route state error

ror. Reading the rows cumulatively makes it possible to identify whether an individual module is useful by itself or only after it is fused with the subsequent estimator.

Table 3. Progressive ablation of the FieldAnchor-LR localization framework. The rows are cumulative rather than independent alternatives: each row retains the components above it and adds the newly named module. Progress MAE is reported in meters along the planned route, and lower values indicate a state that follows the prescribed route progress more closely.

Configuration	MLE	P90	LSR@3	LSR@5	LSR@10	LSR@15	Progress MAE
MS only	4.705	9.072	35.739	57.357	94.992	99.321	10.889
MS + 3-frame GRU	4.795	9.362	33.588	57.838	94.482	99.236	10.914
MS + GRU + inertial polynomial	4.818	9.502	34.946	57.810	94.284	99.293	10.782
FieldAnchor-LR (+ learned -variance KF)	3.482	6.620	49.208	76.995	98.359	99.293	0.195

As shown in Table 3, the largest improvement appears when the temporal motion state is fused with the learned-variance Kalman Filter. Relative to the polynomial-only row, the complete FieldAnchor-LR configuration reduces MLE from 4.818 m to 3.482 m, P90 from 9.502 m to 6.620 m, and Progress MAE from 10.782 m to 0.195 m. The intermediate rows therefore show that the GRU and inertial polynomial mainly provide a structured motion state whose benefit is realized by uncertainty-aware fusion with the visual measurement.

5.4 Hard Forward Candidate Restriction

This experiment evaluates the hard forward candidate restriction that defines the term “Hard” in FieldAnchor-LR and examines whether it can reduce online computation while maintaining localization accuracy. Table 4 compares the complete 6×6 local geometry, which scores all 36 candidates, with the hard causal forward 3×6 geometry, which scores only the 18 candidates in front of the current route direction. The Candidates column is therefore the number of online satellite patches that are projected and scored, MLE/P90/LSR describe localization accuracy, and Latency/FPS describe the end-to-end operating cost under otherwise unchanged model settings.

Table 4. Comparison of complete 6×6 and hard forward 3×6 candidate restriction. Complete 6×6 scores all 36 candidates in the local neighborhood. Hard forward 3×6 retains only the 18 candidates selected by the causal route-aligned projection; all remaining visual, temporal, and filtering settings are unchanged.

Search	Candidates	MLE	P90	LSR@3	LSR@5	LSR@10	LSR@15	Latency (ms)	FPS
Full 6×6	36	3.766	7.225	42.898	69.836	98.727	99.887	69.993	14.287
Forward 3×6 (FieldAnchor-LR)	18	3.501	6.948	49.689	72.864	98.642	100.000	69.490	14.390

As shown in Table 4, Hard forward 3×6 improves MLE from 3.766 m to 3.501 m and P90 from 7.225 m to 6.948 m while reducing the number of scored candidates from 36 to 18. LSR@3 and LSR@5 also increase, and the measured latency changes from 69.993 ms to 69.490 ms. LSR@10 is marginally higher for Complete 6×6 , but both settings are essentially saturated at the relaxed 10–15 m thresholds, indicating that the rear half-grid contributes little useful localization evidence under the tested route-conditioned protocol.

5.5 Temporal Window Length

This experiment verifies how much temporal history is required by the recurrent model. Table 錯誤! 找不到參照來源。 compares one-, two-, and three-frame inputs under the same visual decoder and final estimator. The one-frame setting uses the current observation without explicit temporal differences; the two-frame setting exposes the most recent first difference; and the three-frame FieldAnchor-LR setting makes the temporal mean, first difference, and second difference available to the GRU. Progress MAE is included together with MLE, P90, and LSR to show whether the temporal representation improves not only point accuracy but also along-route state consistency.

Table 5. Effect of the temporal input window. The temporal input changes only the amount of recent visual history supplied to the recurrent model. The three-frame setting corresponds to the complete FieldAnchor-LR temporal representation described in Section 4.6.

Temporal input	MLE	P90	LSR@3	LSR@5	LSR@10	LSR@15	Progress MAE
1 frame	3.919	7.666	44.171	69.128	98.359	99.434	0.201
2 frames	3.936	7.428	44.001	67.968	98.132	99.179	0.306
3 frames (FieldAnchor-LR)	3.482	6.620	49.208	76.995	98.359	99.293	0.195

As shown in Table 錯誤! 找不到參照來源。, the three-frame input obtains the lowest MLE (3.482 m), P90 (6.620 m), and Progress MAE (0.195 m), and it raises LSR@5 to 76.995%. The two-frame setting does not consistently improve over one frame, whereas three frames are the first configuration that exposes both a first temporal difference and a second temporal difference to the GRU. The result is therefore consistent with the temporal representation defined in Equations (11)–(13) and illustrated in Figure 7.

5.6 Second-Order Inertial Motion Model

The motion ablation isolates the effect of the acceleration term in the inertial prediction. Table 錯誤! 找不到參照來源。 compares GRU velocity-only propagation with the second-order velocity-plus-acceleration polynomial used by FieldAnchor-LR while keeping the visual l

ocalization, recurrent structure, and final evaluation protocol unchanged. In addition to MLE, P90, and LSR, Progress MAE measures along-route position consistency and Speed MAE measures the error of the learned per-frame motion, so the table evaluates both localization accuracy and the quality of the propagated motion state.

Table 6. Ablation of the inertial motion prediction. The first row uses the GRU velocity output for propagation. The second row additionally uses the predicted acceleration through the second-order inertial polynomial; Speed MAE evaluates the accuracy of the resulting per-frame motion estimate.

Motion model	MLE	P90	LSR@3	LSR@5	LSR@10	LSR@15	Progress MAE	Speed MAE
GRU velocity only	3.690	7.567	46.972	73.543	98.529	99.547	0.229	0.794
GRU velocity + acceleration polynomial (FieldAnchor-LR)	3.482	6.620	49.208	76.995	98.359	99.293	0.195	0.746

As shown in Table 錯誤! 找不到參照來源。, adding the acceleration term reduces MLE from 3.690 m to 3.482 m, P90 from 7.567 m to 6.620 m, Progress MAE from 0.229 m to 0.195 m, and Speed MAE from 0.794 to 0.746 m per frame. The gains are concentrated in mean error, tail error, strict-threshold success, and motion consistency rather than the already saturated 10–15 m success region. This supports retaining the second-order velocity-plus-acceleration form instead of velocity-only propagation.

5.7 Kalman Filtering and Measurement Variance

This experiment isolates the final state-estimation stage using the configurations available in the versioned FieldAnchor-LR results. Table 7 compares the recurrent visual measurement without Kalman fusion against the complete learned-variance Kalman Filter. Both rows use the same upstream visual and temporal pipeline; the difference is whether the motion prediction and visual measurement are fused using the learned measurement covariance. The comparison therefore measures the contribution of uncertainty-aware Kalman fusion rather than changing the visual search condition.

Table 7. Effect of Kalman filtering and learned measurement variance. No Kalman Filter uses the recurrent visual measurement directly. The FieldAnchor-LR row fuses the same upstream measurement with the inertial prediction using the learned measurement covariance and the Kalman update defined in Equations (19)–(21).

Final estimator	MLE	P90	LSR@3	LSR@5	LSR@10	LSR@15
No Kalman Filter	4.818	9.502	34.946	57.810	94.284	99.293
Learned-variance KF (FieldAnchor-LR)	3.482	6.620	49.208	76.995	98.359	99.293

As shown in Table 7, the learned-variance Kalman Filter reduces MLE from 4.818 m to 3.482 m and P90 from 9.502 m to 6.620 m. LSR@3 rises from 34.946% to 49.208% and LSR@5 from 57.810% to 76.995%, whereas LSR@15 is unchanged because both configurations already place nearly all frames inside the broad 15 m radius. The filter therefore improves the precision of moderate-error frames through covariance-aware fusion of the visual measurement and inertial prediction.

5.8 Route-Wise Final Localization Results

The complete FieldAnchor-LR model is evaluated separately on held-out Routes B and C, and Table 8 then aggregates all held-out frames in the B + C row. The MLE and Median columns describe central localization error, P90 and P95 expose the tail of the error distribution, and LSR@3 through LSR@20 report the fraction of frames inside increasingly relaxed metric radii. Presenting the two routes separately is important because similar mean errors can still hide different median and tail behavior.

Table 8. Final FieldAnchor-LR localization results on held-out routes. Route B and Route C are reported independently, and B + C pools all 3,534 held-out frames. Lower MLE/Median/P90/P95 and higher LSR indicate better localization performance.

Route	MLE	Median	P90	P95	LSR@3	LSR@5	LSR@10	LSR@15	LSR@20
Route B	3.437	3.272	6.512	7.633	45.387	76.582	99.605	100.000	100.000
Route C	3.565	2.624	6.909	8.374	56.121	77.742	96.105	98.013	99.205
B+C (all frames)	3.482	3.070	6.620	7.847	49.208	76.995	98.359	99.293	99.717

As shown in Table 錯誤! 找不到參照來源。 , the complete model reaches an aggregate MLE of 3.482 m, a median error of 3.070 m, P90 of 6.620 m, and P95 of 7.847 m across the 3,534 held-out frames. Route B and Route C have similar mean errors (3.437 m and 3.565 m), but their distributions differ: Route C has the lower median and higher LSR@3, while Route B has the lighter tail and higher success rates at the relaxed 10–20 m thresholds. Reporting the routes separately therefore reveals behavior that would be hidden by the B+C average alone.

5.9 Comparison with Existing UAV–Satellite Localization Methods

The external comparison follows each method’s native localization protocol when a versioned output is available. Table 錯誤! 找不到參照來源。 therefore includes a Native protocol column before the numerical metrics: DenseUAV [7], Sample4Geo [4], and Game4Loc [8] use fixed-gallery global retrieval, InfoGeo [10] uses global retrieval when a versioned output is available, Bearing-UAV [9] is listed as neighbor-map regression, and FieldAnchor-LR uses the controlled local-search protocol described in this thesis. MLE, Median, P90, P95, and LSR provide the common error scale, while FPS gives the measured throughput when available. An em dash indicates that a versioned official-protocol result is not available in the current result package; it is not treated as a zero or estimated value.

Table 9. Comparison with external UAV–satellite localization methods on FieldAnchor. The Native protocol column must be read together with the metrics because global retrieval, neighbor-map regression, and controlled local search have different search scopes. The table is therefore used for protocol-aware error-scale context rather than a strictly matched architecture ranking.

Method	MLE	Median	P90	P95	LSR@3	LSR@5	LSR@10	LSR@15	FPS
DenseUAV [7]	417.282	347.952	872.145	1073.156	0.255	0.623	1.019	1.868	99.677
Sample4Geo [4]	284.905	241.941	588.841	709.878	0.340	1.330	2.320	3.537	226.264
Game4Loc [8]	184.182	124.480	434.277	559.439	2.235	5.065	7.838	12.054	229.348
InfoGeo [10]	238.403	132.820	541.617	731.367	0.509	1.500	4.160	7.612	—†
Bearing-UAV [9]	215.277	164.577	461.801	542.675	0.057	0.113	0.340	0.990	98.569
FieldAnchor-LR (Ours)	3.482	3.070	6.620	7.847	49.208	76.995	98.359	99.293	15.137

As shown in Table 錯誤! 找不到參照來源。, the available fixed-gallery global-retrieval outputs have a substantially larger error scale on FieldAnchor than the controlled local-relocalization setting used by FieldAnchor-LR. This numerical gap must be read together with the Native protocol column: the methods do not operate under the same search scope, so Table 錯誤! 找不到參照來源。 is not used to claim a direct architecture-only ranking. InfoGeo and Bearing-UAV remain marked with an em dash until versioned outputs under their documented protocols are available.

In addition to the protocol-aware numerical comparison in Table 9, a qualitative cross-method comparison is reserved in Figure 10. The final version should reproduce outputs from the available external methods on the same FieldAnchor queries and visualize each method’s predicted position together with the corresponding predefined route reference point and metric error. Because the compared methods retain their native search protocols, this figure is intended to expose characteristic success and failure cases rather than to imply a strictly matched architecture ranking.



Figure 10. Cross-method qualitative localization results on FieldAnchor.

For the final submission, each panel in Figure 10 should use the same held-out UAV frame and map region for all reproduced methods. A compact and interpretable layout is to show the UAV query, the method prediction on the orthomosaic, the predefined route reference point, and the localization error in meters. This keeps the qualitative comparison aligned with Table 9 while avoiding an unfair visual comparison caused by using different query frames or map extents.



5.10 Backbone Accuracy–Runtime Trade-off on NVIDIA Jetson Orin NX

To evaluate the deployment cost of the visual encoder, four backbones are benchmarked on an NVIDIA Jetson Orin NX development platform under the same backbone-comparison procedure. MobileNetV3-Small, ResNet-18, VGG16, and MobileCLIP2-S2 are evaluated on the held-out B+C split. End-to-end (E2E) average latency and P95 latency report complete per-frame inference time for this benchmark on the Orin NX platform, approximate FPS summarizes throughput, and B+C MLE measures localization accuracy. The current benchmark record does not specify the power mode, numerical precision, or acceleration backend, so those settings are not inferred here.

Table 10. Backbone accuracy–latency trade-off on the NVIDIA Jetson Orin NX development platform. All latency and throughput values in Table 錯誤! 找不到參照來源。 are measured on the same NVIDIA Jetson Orin NX development platform. Lower E2E latency, P95 latency, and MLE are better, whereas higher FPS is better. The B+C MLE values belong specifically to this backbone deployment sweep and should therefore be interpreted within this comparison rather than substituted for the final aggregate result reported in Table 8.

Backbone	E2E mean latency	P95 latency	Approx. FPS	B+C MLE
MobileNetV3-Small	88.83 ms	91.60 ms	11.26	4.224 m
ResNet-18	95.07 ms	97.85 ms	10.52	4.077 m
VGG16	137.35 ms	147.53 ms	7.28	4.277 m
MobileCLIP2-S2	188.55 ms	191.44 ms	5.30	3.634 m

As shown in Table 錯誤! 找不到參照來源。 , the backbone benchmark exhibits a clear accuracy–throughput trade-off on the Jetson Orin NX. MobileNetV3-Small is the fastest configuration, with 88.83 ms average E2E latency, 91.60 ms P95 latency, and approximately 11.26 FPS, but its B+C MLE is 4.224 m. ResNet-18 retains similar throughput at 10.52 FPS while reducing MLE to 4.077 m. VGG16 is slower and, in this experiment, also has the highest MLE among the four backbones, with 137.35 ms average latency and 4.277 m MLE. MobileCLIP2-S2 provides the lowest B+C MLE of 3.634 m, but it also has the largest runtime cost, reaching 188.55 ms average latency, 191.44 ms at P95, and approximately 5.30 FPS. Relative to MobileNetV3-Small, MobileCLIP2-S2 reduces MLE by 0.590 m (approximately 14.0%) while requiring about 2.12× the average latency. These results indicate that the visual backbone should be selected as an accuracy–runtime trade-off for embedded deployment: lightweight CNNs offer higher on-device throughput, whereas MobileCLIP2-S2 provides the strongest localization accuracy in the measured Orin NX backbone sweep.

5.11 Qualitative Trajectory Analysis

Two qualitative views are used to complement the scalar metrics. Figure 11 is reserved for a contiguous held-out sequence that exposes frame-to-frame temporal stability, while Figure 12 overlays the final filtered route on the FieldAnchor orthomosaic to show route-level continuity.

Figure 11 should visualize one uninterrupted sequence rather than isolated frames. The recommended plot places frame index on the horizontal axis and either localization error or along-route progress on the vertical axis, with curves for MS-only, MS + three-frame GRU, MS + GRU + second-order inertia, and the complete FieldAnchor-LR estimator. This directly shows whether the recurrent and filtering components reduce jitter, overshoot, and abrupt one-frame corrections that may not be visible from MLE or P90 alone.

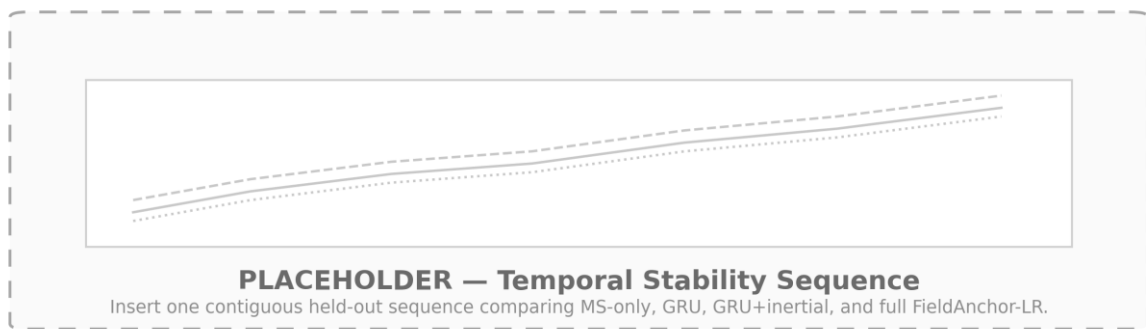


Figure 11. Temporal stability on a contiguous held-out FieldAnchor sequence. The final Figure 11 should use exactly the same frame interval for every configuration and should not smooth the plotted outputs for visualization. If along-route progress is used, the registered route progress can be included as a reference curve; if localization error is used, the plot should retain the original per-frame values so that transient spikes remain visible. The purpose is to demonstrate temporal behavior, not to duplicate the aggregate values already reported in Tables 3, 5, 6, and 7.

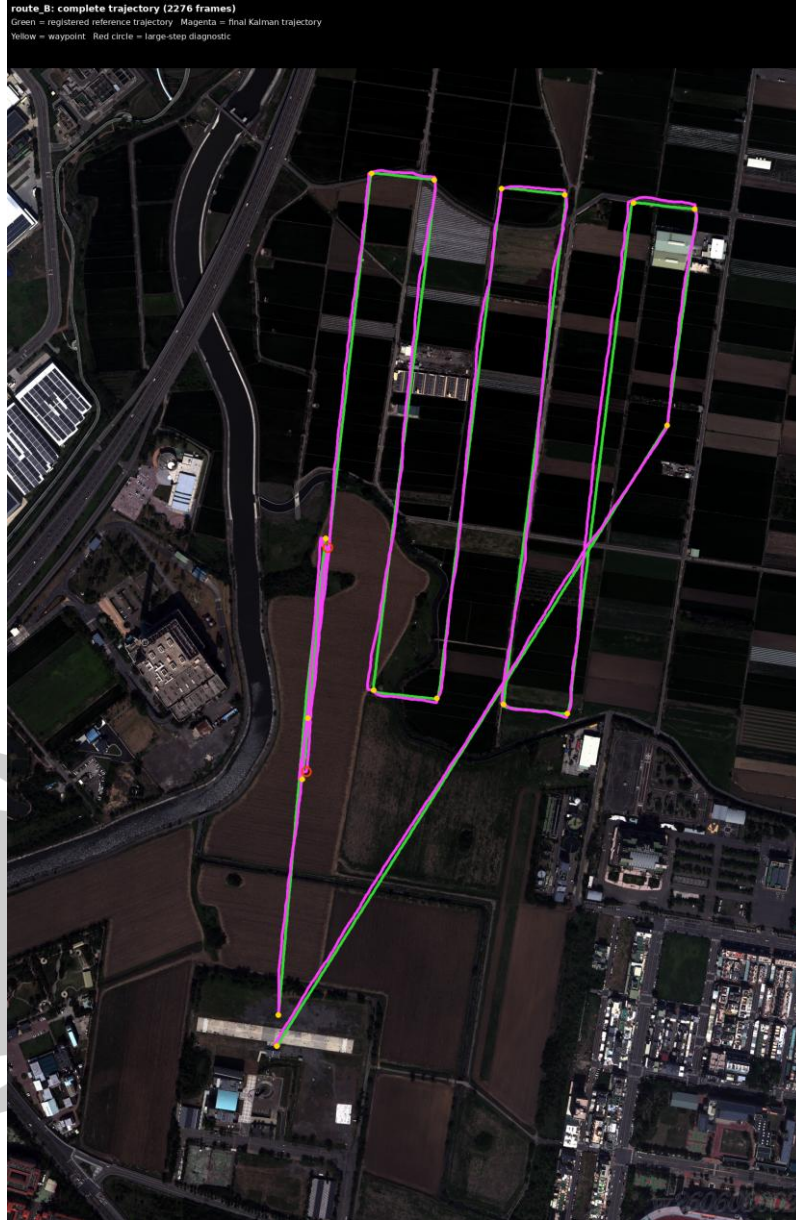


Figure 12. Final FieldAnchor-LR trajectory on the FieldAnchor orthomosaic. Figure 12 overlays the planned reference route and the final filtered trajectory to provide a route-level view of continuity that is not captured by scalar error statistics alone. The filtered path should be interpreted together with Figure 11: Figure 11 exposes local frame-to-frame stability, whereas Figure 12 shows whether those local updates remain coherent over the complete held-out route. Detailed quantitative conclusions remain in Tables 3–9.

Along straight route segments, the filtered estimate changes smoothly because the inertial prediction supplies a causal motion prior and the Mean-Shift observation supplies an uncertainty-quantified visual measurement. This behavior is important in repetitive farmland, where several neighboring satellite crops can receive similar visual responses even when their metric centers differ.

Near route turns, the recurrent model adapts the local motion trend through the estimated heading, velocity, and acceleration states, while the learned measurement covariance adjusts the relative contribution of the current visual observation in the Kalman gain. The temporal model and uncertainty-aware filter therefore address motion continuity and visual ambiguity without changing the definition of “Hard,” which remains exclusively the forward 3×6 candidate restriction.

The trajectory visualization is consequently used only for qualitative continuity analysis. Quantitative conclusions about component contributions, search geometry, temporal window length, motion prediction, Kalman fusion, and route-wise accuracy are reported once in Tables 錯誤! 找不到參照來源。—錯誤! 找不到參照來源。 and discussed directly beside those tables, avoiding duplicate bar charts or curves that repeat the same numerical evidence.



Chapter 6 Conclusion

This thesis presents FieldAnchor and the FieldAnchor Local Relocalization (FieldAnchor-LR) framework for route-conditioned UAV–satellite visual relocalization in repetitive farmland. The complete method combines a frozen MobileCLIP2-S2 representation, a controlled local-search condition, causal forward 3×6 candidate selection, continuous Mean-Shift (MS) visual localization, three-frame Gated Recurrent Unit (GRU) temporal modeling, a second-order inertial polynomial, learned visual-measurement variance, and an external uncertainty-aware Kalman Filter (KF) in continuous route coordinates. The “Hard” wording in the fixed thesis title refers exclusively to the explicit forward candidate restriction that retains the route-aligned 3×6 subset from the complete 6×6 local geometry before Mean-Shift decoding; it does not denote a separate MS variant.

On the held-out Routes B and C, the complete FieldAnchor-LR configuration achieves an overall Mean Localization Error of 3.482 m, a median error of 3.070 m, and a P90 of 6.620 m. The combined Localization Success Rates are 76.995% within 5 m, 98.359% within 10 m, and 99.293% within 15 m. The ablation results show that the learned-variance Kalman Filter provides the largest improvement in final localization accuracy, while the three-frame representation and second-order motion model provide the temporal state used by the final fusion stage. The hard forward 3×6 candidate restriction halves the number of online satellite candidates from 36 to 18 before visual scoring and Mean-Shift decoding, while the subsequent MS, GRU, inertial, and Kalman stages operate as separate components of the localization pipeline. The reported experiment is a route-conditioned local relocalization study around predefined planned-route reference points and does not claim unrestricted global acquisition from an unknown map position.

References

- [1] S. Hu, M. Feng, R. M. H. Nguyen, and G. H. Lee, “CVM-Net: Cross-View Matching Network for Image-Based Ground-to-Aerial Geo-Localization,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7258–7267, 2018.
- [2] Y. Shi, L. Liu, X. Yu, and H. Li, “Spatial-Aware Feature Aggregation for Image-Based Cross-View Geo-Localization,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [3] S. Zhu, M. Shah, and C. Chen, “TransGeo: Transformer Is All You Need for Cross-View Image Geo-Localization,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1162–1171, 2022.
- [4] F. Deuser, K. Habel, and N. Oswald, “Sample4Geo: Hard Negative Sampling for Cross-View Geo-Localisation,” *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 16847–16856, 2023.
- [5] Z. Zheng, Y. Wei, and Y. Yang, “University-1652: A Multi-View Multi-Source Benchmark for Drone-Based Geo-Localization,” *Proceedings of the 28th ACM International Conference on Multimedia*, pp. 1395–1403, 2020.
- [6] R. Zhu, L. Yin, M. Yang, F. Wu, Y. Yang, and W. Hu, “SUES-200: A Multi-Height Multi-Scene Cross-View Image Benchmark Across Drone and Satellite,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 33, no. 9, pp. 4825–4839, 2023.
- [7] M. Dai, E. Zheng, Z. Feng, L. Qi, J. Zhuang, and W. Yang, “Vision-Based UAV Self-Positioning in Low-Altitude Urban Environments,” *IEEE Transactions on Image Processing*, vol. 33, pp. 493–508, 2024.
- [8] Y. Ji, B. He, Z. Tan, and L. Wu, “Game4Loc: A UAV Geo-Localization Benchmark from Game Data,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 4, pp. 3913–3921, 2025.
- [9] K. Liu, H. Zhou, R. Xu, P. Wang, M. Song, and H. Zhang, “Beyond Matching to Tiles: Bridging Unaligned Aerial and Satellite Views for Vision-Only UAV Navigation,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5359–5368, 2026.

- [10] H. Zhang, M. Wang, Z. Wang, H. Yin, and M. O. Pun, “InfoGeo: Information-Theoretic Object-Centric Learning for Cross-View Generalizable UAV Geo-Localization,” *Proceedings of the 43rd International Conference on Machine Learning*, PMLR 306, 2026.
- [11] D. Comaniciu and P. Meer, “Mean Shift: A Robust Approach Toward Feature Space Analysis,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 5, pp. 603–619, 2002.
- [12] F. Faghri, P. K. A. Vasu, C. Koc, V. Shankar, A. Toshev, O. Tuzel, and H. Pouransari, “MobileCLIP2: Improving Multi-Modal Reinforced Training,” *Transactions on Machine Learning Research*, 2025.
- [13] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,” *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pp. 1724–1734, 2014.
- [14] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [15] J.-W. Hsieh, C.-H. Chou, M.-C. Chang, P.-Y. Chen, S. Santra, and C.-S. Huang, “Mean-Shift Based Differentiable Architecture Search,” *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 3, pp. 1235–1246, Mar. 2024, doi: 10.1109/TAI.2023.3329792. DOI
- [16] Z. Li, X. Zhang, Y. Guo, Y. Xie, Z. Ding, S. Cai, H. Li, and M. Lao, “PAUL: Uncertainty-Guided Partition and Augmentation for Robust Cross-View Geo-Localization under Noisy Correspondence,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5389–5398, 2026.
- [17] X. Liang, H. Tang, F. Zhang, S. Yuan, C. Hu, D. Zheng, and K. Ma, “UniGeoRS: A Unified Benchmark for Tri-view Geo-Localization,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5399–5408, 2026.
- [18] Z. Song, J. Zhang, D. Wang, Z. Zhou, W. Liu, H. Guo, E. Wang, and B. Du, “GeoBridge: A Semantic-Anchored Multi-View Foundation Model Bridging Images and Text for Geo-Localization,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 27793–27803, 2026.

- [19] X. Cheng, L. Wang, Y. Liu, X. Liu, H. Tan, Y. Liu, M. Zhang, and S. Yan, “PiLoT: Neural Pixel-to-3D Registration for UAV-based Ego and Target Geo-localization,” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5379–5388, 2026.
- [20] Y. Zhang, X. Zhang, G. Sun, Z. Lyu, S. Wshah, and C. Chen, “Geo2: Geometry-Guided Cross-view Geo-Localization and Image Synthesis,” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 19432–19442, 2026.

